
Real Estate Management

Release 8.0

jankstar

Sep 18, 2026

CONTENTS

1	Setup	3
2	Configuration	5
2.1	WoWi Chart of Accounts [auto]	5
2.2	Account Configuration	5
2.3	Use Classes [auto]	6
2.4	Measurement Types [auto]	6
2.5	Object Party Roles [auto]	7
2.6	Contract Types [auto]	7
2.7	Contract Term Types [auto]	7
2.8	Cost Category Groups [auto]	8
2.9	Cost Types (§ 2 BetrKV) [auto]	8
2.10	Taxes	8
3	Usage	9
3.1	Step 1 — Create a Property	9
3.2	Step 2 — Enter Measurements	10
3.3	Step 3 — Assign Partners and Roles	10
3.4	Step 4 — Create Contracts	10
3.5	Step 5 — Generate Invoices (CreateContractMoves)	11
3.6	Step 6 — Meter Readings	12
3.7	Step 7 — Operating Cost Settlement (Billing Unit)	12
3.8	Typical Annual Workflow Summary	15
4	Design	17
4.1	Object Hierarchy	17
4.2	Measurement Group Hierarchy	17
4.3	Contract Architecture	18
4.4	Contract Workflow	18
4.5	Occupancy Tracking	19
4.6	Operating Cost Settlement Pipeline	19
4.7	Allocation Rules	20
4.8	Invoice Integration	20
4.9	Tryton Framework Patterns Used	21
5	API Reference	23
5.1	Property Management	23
5.2	Contract Management	27
5.3	Operating Cost Settlement	30
5.4	Extensions to Core Modules	33

5.5 Wizards	33
6 Release notes	35
7 Module Dependencies	37
8 Installation	39
9 Configuration	41
10 Access Control	45
11 Data Model	49
11.1 Property Management	49
11.2 Contract Management	52
11.3 Operating Cost Settlement	55
11.4 CO2 Cost Allocation (CO2KostAufG)	61
11.5 BVED External Billing Interface	63
11.6 Option Rate (Input VAT Deduction)	73
11.7 Extensions to Core Modules	75
12 Wizards	81
13 Reports	85
14 Accounting / WoWi	87
15 Source Layout	89
16 Running Tests	91
16.1 Demo data scripts	91

A Tryton ERP module for real estate management. Packaged as `jankstar_real_estate`, targeting Tryton 8.0.0 and Python 3.9–3.13.

Covers property management, lease and sales contracts, tenant/owner party roles, operating cost settlement, CO2 cost allocation under the German CO2KostAufG, and a German “Kontenrahmen der Wohnungswirtschaft” (WoWi) accounting chart.

ISO_4217 Currency https://de.wikipedia.org/wiki/ISO_4217

ISO_3166 Country Codes https://www.laenderdaten.de/kuerzel/iso_3166-1.aspx

This module was developed with the support of AI cloude code — all requirements, specifications, tests, corrections, and optimizations were carried out by human contributors.

CHAPTER
ONE

SETUP

CONFIGURATION

This page describes every master-data record that must exist before the `real_estate` module can be used productively. Records marked **[auto]** are created automatically when the module is installed via `trytond-admin -u real_estate`. All other records must be set up manually or confirmed after installation.

2.1 WoWi Chart of Accounts [auto]

The module ships a complete German “Kontenrahmen der Wohnungswirtschaft” (WoWi, housing-industry chart of accounts) under `wowi/`. It is loaded automatically at installation and provides:

- Account types and accounts (e.g. 6000 Sollmieten, 2000 Mietenkontokorrent-Sammelkonto)
- Tax groups, tax templates (7 % / 19 % MwSt.)
- Tax code templates and tax rule templates

After installation, open *Accounting* ▢ *Configuration* ▢ *Account Templates* and apply the chart to your company (button *Create Chart of Account*) if it has not been applied yet.

2.1.1 Rental Book Journal [auto]

A journal named **Rental Book** (code RE, type *Revenue*) is created automatically. It is used as the default posting journal for all contract types. If you prefer a different journal, you can change it on each contract type individually.

2.2 Account Configuration

Open *Accounting* ▢ *Configuration* ▢ *Configuration* and fill in the real-estate-specific defaults (added by this module via `account_configuration.py`, model `account.configuration.real_estate`):

Vacancy Cost Account (re_account_allocation_by_owner)

Account used for direct GL postings of vacancy settlement results (cost shares with no contract, i.e. the owner’s share of unoccupied periods) — both the debit and credit side of the posting.

Operating Cost Settlement Journal (re_journal_billing)

Journal used for the vacancy GL postings above.

Operating Cost Billing Payment Term (re_payment_term_billing)

Default payment term for operating cost settlement invoices, used whenever the contract itself has no payment term set.

Vacancy Cost Account and Operating Cost Settlement Journal are required before vacancy results can be billed (see `BillingUnit.billing_wizard / billing` in `billing_unit.py`); tenant invoices are unaffected and use the contract's own accounts/journal.

2.3 Use Classes [auto]

Six use classes are loaded automatically:

Seq.	Name	Basement No.	Parking No.
10	Apartment	yes	—
20	Office	yes	—
30	Retail	yes	—
40	Warehouse	yes	—
50	Parking	—	yes
60	Garage	—	yes

Additional classes can be created under *Real Estate* ▢ *Configuration* ▢ *Use Classes* without changing any code. The `has_basement_nr` / `has_parking_nr` flags control which extra fields appear on the rental-object form.

2.4 Measurement Types [auto]

The following measurement types are loaded automatically:

Seq.	Name	Unit	Applies to	Group
05	Usable Space	m ²	object	yes (root)
10	Living Space	m ²	object	child of <i>Usable Space</i>
15	Commercial Space	m ²	object	child of <i>Usable Space</i>
20	Number of rooms	unit	object	—
30	Gross floor area	m ²	building	—
40	Land area	m ²	land	—
50	Number of items	unit	equipment	—
60	Property value	EUR	property	—

Usable Space is a group type: when it is referenced in a term type or settlement unit, the system automatically includes measurements of all child types (*Living Space*, *Commercial Space*) in the calculation.

Additional types can be created under *Real Estate* ▢ *Configuration* ▢ *Measurement Types*. All children of a group must share the same unit; circular parent references are rejected.

2.5 Object Party Roles [auto]

Three roles are loaded automatically:

Seq.	Name	Applies to object types
10	Caretaker (default)	property
20	Administrator	property
30	Owner	property, object

Additional roles can be created under *Real Estate* ▢ *Configuration* ▢ *Object Party Roles*.

2.6 Contract Types [auto]

Six contract types are loaded automatically:

Seq.	Name	Prefix	Direction	Occu- pancy	Type of use
10	Rental agreement	1	Debit (out)	yes	residential
20	Parking Space Lease Agreement	2	Debit (out)	yes	residential
30	Debit Costs Agreement	3	Debit (out)	—	all
40	Credit Costs Agreement	4	Credit (in)	—	all
50	Commercial lease agreement	5	Debit (out)	yes	commer- cial
80	Condominium fee agreement	8	Debit (out)	yes	property

For each contract type you should check and set, if not already done:

- **Default account** — the revenue/expense account for invoice lines
- **Default taxes** — e.g. 19 % USt. for commercial contracts
- **Payment term** — optional default payment term

2.7 Contract Term Types [auto]

Seven term types are loaded automatically:

Seq.	Name	Rhythm	Type of use / m_type
1000	Apartment rent (monthly)	1 × monthly	residential / Living Space
1100	Parking space rent (monthly)	1 × monthly	residential, commercial, internal / —
2000	AP for Operating costs (monthly)	1 × monthly	all / Living Space
3000	AP for Heating costs (monthly)	1 × monthly	all / Living Space
8000	Commercial rent (monthly)	1 × monthly	commercial / Commer- cial Space
9000	Condominium fees (monthly)	1 × monthly	property / —

For each term type you can optionally set:

- **Default account** — pre-filled on the contract term when this type is selected
- **m_type** — measurement type used to derive the default quantity (the value is looked up from the referenced contract item's objects)

2.8 Cost Category Groups [auto]

Five groups are loaded automatically, following the BetrKV paragraph structure:

- I · Grundbesitzabgaben & Versicherungen (seq. 100)
- II · Ver- und Entsorgung (seq. 200)
- III · Reinigung, Pflege & Sicherheit (seq. 300)
- IV · Sonstige Betriebskosten (seq. 400)
- V · Keine Umlage (seq. 900)

2.9 Cost Types (§ 2 BetrKV) [auto]

The following cost types are loaded automatically, covering the standard positions of § 2 BetrKV:

Seq.	Name (§ 2 BetrKV)	Group
100	Grundsteuer (Nr. 1)	I
110	Gebäudeversicherung (Nr. 13)	I
200	Wasserversorgung, Abwasser (Nr. 2 + 3)	II
300	Heizung – Brennstoff (Nr. 4)	II
310	Heizung – Wartung (Nr. 4)	II
320	Warmwasser (Nr. 5)	II
330	Verbundene Heizungs- und Warmwasserversorgung (Nr. 6)	II
400	Aufzug (Nr. 7)	II
500	Straßenreinigung (Nr. 8)	III
510	Müllabfuhr (Nr. 8)	III
520	Hausreinigung (Nr. 9)	III
600	Gartenpflege (Nr. 10)	III
610	Hausstrom (Nr. 11)	III
620	Schornsteinfeger (Nr. 12)	III
700	Hausmeister (Nr. 14)	IV

Additional cost types (e.g. antenna, broadband, communal laundry) can be added freely under *Real Estate* ☐ *Configuration* ☐ *Cost Types*.

2.10 Taxes

The WoWi templates include tax records for 7 % and 19 % VAT. After applying the chart of accounts, verify that the tax record *USt. 19 % Umsatzsteuer voller Satz Waren Inland* exists under *Accounting* ☐ *Taxes*. This tax is used by `test_contracts.py` for commercial leases and should be assigned to the *Commercial lease agreement* contract type as a default tax.

USAGE

This page describes the end-to-end workflow for the `real_estate` module, from setting up a property to running the annual operating cost settlement. All steps assume the configuration described in [Configuration](#) has been completed.

3.1 Step 1 — Create a Property

Navigate to *Real Estate* ▢ *Properties*.

1. Click **New** and set **Type** to *Property*.
2. Fill in **Name**, **Company**, **Start date**, and optionally assign an **Address** (select from *Real Estate* ▢ *Master Data* ▢ *Addresses*).
3. Set **Type of use** (e.g. *Residential*) — this filters which contract types are available later.
4. Optionally set **Billing as** and **Collective billing** to control how operating cost billing is aggregated across buildings.
5. Click **Approved** to transition the property from *Draft* to *Approved* (button on the form toolbar). Further transitions are *Approved* ▢ *Locked* ▢ *Deactivated* (and back from *Locked* to *Approved*, or *Approved* to *Draft*).

Under the property you can then create the sub-objects of the tree:

Type	Parent	Typical use
Building	Property	Multi-unit residential or commercial building
Land	Property	Plot without building (e.g. parking lot)
Object	Building or Land	Rental unit: apartment, retail space, parking space
Equipment	Building, Land, Object, or Equipment	Meters (water, heat), technical installations

For each **Object** set the **Use class** (Apartment, Retail, Parking, ...) and optionally enter the **Object number** (shown on invoices and reports).

For each **Equipment** of e_type *Meters* set:

- **Meter unit** — the UOM of the readings (e.g. m³)
- **Meter factor** — conversion factor (usually 1.0)
- **No decimals** — tick if readings must be integer values
- **Meter ID** — identifier printed on the physical meter

Activate each object with the **Active** button after completing it.

3.2 Step 2 — Enter Measurements

Measurements record the physical size of an object for a given date. They are used as the allocation base for operating cost settlement (e.g. living area in m²) and as the default quantity on contract terms.

Navigate to the object's form and open the **Measurements** tab, or use *Real Estate* ▢ *Master Data* ▢ *Measurements*.

For each rental object enter at least:

- **Living Space** (m²) — used by *Apartment rent* and *Operating costs* term types as default quantity
- **Number of rooms** — optional, for reports

For commercial objects enter:

- **Commercial Space** (m²) — used by *Commercial rent* term types

For buildings:

- **Gross floor area** (m²) — used in billing unit reports

The **Valid from** date determines from when the measurement applies. Multiple measurements of the same type can coexist if the area changes over time; the system always uses the latest measurement valid on or before the reference date.

3.3 Step 3 — Assign Partners and Roles

Open the **Parties** tab on any property or object to assign party . party records with a role (e.g. Owner, Administrator).

For owners this is informational; for tenants the party is entered directly on the contract (Step 4).

3.4 Step 4 — Create Contracts

Navigate to *Real Estate* ▢ *Contracts*.

1. Click **New** and set:

- **Contract type** — e.g. *Rental agreement* (residential)
- **Property** — the parent property
- **Contractual partner** — the tenant (party . party)
- **Invoice address** — the partner's delivery/invoice address
- **Start date** — first day of the tenancy
- **Currency** — filled automatically from the company

2. On the **Items** tab, click **New** to add a ContractItem:

- **Valid from** — typically the contract start date
- Add one or more rental objects in the **Objects** sub-table. The object selector is restricted to objects of type *object* that belong to the same property as the contract.

3. On the **Terms** tab, click **New** to add each charge (ContractTerm):

- **Term type** — e.g. *Apartment rent* (monthly)
- **Reference item** — the contract item this term refers to

- **Valid from** — start of the charge period
- **Rhythm / Rhythm type** — billing frequency (e.g. 1 × monthly)
- **Quantity** — auto-filled from the measurement type linked to the term type (can be overridden)
- **Unit price** — amount per unit per period
- **Taxes** — pre-filled from the contract type defaults

Typical term sequence for a residential contract:

Seq.	Term type	Note
10	Apartment rent (monthly)	Net rent
20	AP for Operating costs (monthly)	Cold costs advance
30	AP for Heating costs (monthly)	Heating advance

4. Click **Running** to activate the contract. The state changes from *Draft* to *Running*.

3.4.1 Terminating a contract

Click **Terminate** on a running contract. The wizard asks for:

- **Terminated by** — tenant or landlord
- **Receipt of termination notice** — date the notice was received
- **Termination reason** — free text
- **Termination date** — last day of the tenancy (calculated from the notice period or entered manually)

The contract moves to *Terminated*. Occupancy records are updated automatically; the object becomes available for a new contract.

3.4.2 Cancelling a contract

The **Cancel** button is available on *Draft* contracts. If cash flow entries in state *done* (already invoiced) exist, a warning is shown: existing postings will **not** be reversed automatically. Draft cash flow entries are deleted automatically on cancellation.

3.5 Step 5 — Generate Invoices (CreateContractMoves)

Navigate to *Real Estate* to *Create Contract Moves* or click the **Create Moves** button on a contract.

The wizard offers three actions:

Create

Generate invoices up to a given cut-off date. For each contract term with pending future cash flow entries up to the date, one invoice line is created and the cash flow entry transitions to *done*.

Re-calc

Rebuild the cash flow projection without creating invoices. Useful after changing a term's rhythm, unit price, or validity dates.

Re-calc and Create

Rebuild first, then create invoices in one step.

You can filter by **Property** and/or **Contract** to limit the run to a subset of contracts.

After the wizard completes, the generated invoices appear under *Accounting* ▢ *Invoices* and can be posted and sent from there.

The **Cash Flow** tabs on the contract form show:

- **Draft** — planned entries not yet invoiced
- **Pending** — posted (open) invoices
- **Paid** — settled invoices

3.6 Step 6 — Meter Readings

Navigate to *Real Estate* ▢ *Master Data* ▢ *Meter Readings*, or open the **Meters** tab on an equipment object.

Reading types:

initial

First reading when the meter is commissioned (or a new tenant moves in). Must be the first reading for a given meter.

reading

Periodic reading. The meter ID must match the previous reading. For counter meters, the **Consumption** is computed automatically (current value minus previous value).

estimate

Created by the **Estimate Consumption** wizard on the equipment form. The wizard interpolates between surrounding readings or extrapolates from the last two readings within one year.

final

Last reading when a meter is decommissioned or replaced.

The **Meter ID** groups readings that belong to the same physical meter. When a meter is replaced, a new initial reading with a different Meter ID starts a new series.

3.7 Step 7 — Operating Cost Settlement (Billing Unit)

The annual settlement is managed through *Billing Units*. Navigate to *Real Estate* ▢ *Operation Costs* ▢ *Billing Units*.

3.7.1 7.1 Create a Billing Unit

1. Click **New** and set:

- **Property** — the property to settle
- **Start date** — first day of the settlement period (e.g. 01.01.2025)
- **End date** — last day (e.g. 31.12.2025); leave blank for open-ended
- **Calculation method** — *Rental apartment* (§§ 1–2 BetrKV) or *WEG billing* (condominium law)
- **Description** — e.g. “Kalte Betriebskosten 2025”
- **Term types of use** — which contract term types count as advance payments (e.g. *AP for Operating costs*)

2. Click **Approved** to activate the billing unit.

3.7.2 7.2 Add Settlement Units

Under the **Settlement Units** tab, add one row per cost type:

- **Cost type** — e.g. *Grundsteuer* (seq. 100)
- **Allocation rule:**
 - allocation_by_measurement**
Allocated proportionally to a measurement value (e.g. living area). Set **Measurement type** — e.g. *Usable Space* (group, expands to Living Space + Commercial Space automatically).
 - allocation_by_consumption**
Allocated by meter reading consumption (e.g. water m³). Set **Meter unit** and **Meter regex** to filter the relevant meters.
 - allocation_per_rental_unit**
Allocated proportionally to occupancy duration.
 - allocation_from_external_billing**
No allocation calculation; amounts are entered manually from an external bill (e.g. heat cost allocator result).
 - no_allocation**
Entire cost stays unallocated (e.g. maintenance reserve).
- **Object regex** — regular expression to filter which rental objects are included (e.g. *Wohnung|Einzelhandel*).
- **Vacancy** — how to handle unoccupied periods: *by_owner* (costs go to the owner) or *unallocated*.

3.7.3 7.3 Run Selection

Click **Selection** to identify which contracts and objects fall within the settlement period. The system creates `CostShare` records for every object × settlement unit combination.

Selection can be re-run from the *Value Share* state to revise the scope; existing settlement results are deleted after confirmation.

3.7.4 7.4 Compute Value Shares

Click **Compute Value Shares** to calculate the allocation factor for each cost share:

- Measurement-based: $\text{value_share} = \text{measurement} \times \text{time_share} / \text{time_total}$
- Consumption-based: $\text{value_share} = \text{consumption}$
- Per rental unit: $\text{value_share} = \text{time_share} / \text{time_total}$

Cost shares that cannot be computed (e.g. missing meter reading) are set to state *error* with an explanatory message. Fix the underlying data and click **Compute Value Shares** again — error shares are automatically reset before the recalculation.

3.7.5 7.5 Compute Settlement Result

Click **Compute Settlement Result** to aggregate cost shares per contract and produce one `SettlementResult` per contract:

- **Planned costs** — budgeted amount (from cost share)
- **Actual costs** — real costs entered or allocated

- **Advanced payment** — sum of *posted* and *paid* invoice lines for the term types of use during the settlement period
- **Refund / receivable** = actual costs – advanced payment (positive = refund to tenant, negative = additional charge)

If any cash flow entries in the settlement period are still in state *draft* or *validated* (not yet posted), the affected cost share is marked *error* with a [draft] prefix. Post or delete those invoice drafts and re-run.

3.7.6 7.6 Ready for Billing

Click **Ready for Billing** to transition from *Value Share* to *Ready for Billing*. This validates, for every settlement unit and settlement result of the billing unit, that:

- no settlement unit is in an error state;
- every settlement unit (except *No allocation*) has completed its value share calculation;
- at least one approved settlement result exists and none has actual costs of zero;
- every settlement result with a non-zero advance payment has both a term and a contract assigned (a missing term usually means several term types contributed and the billing unit's term-type filter needs narrowing);
- all advance-payment invoices are posted;
- unless **External billing** is enabled, the sum of settlement result actual costs matches the sum of cost share actual costs.

Any failed check raises a validation error listing the offending records — fix the underlying data and click **Ready for Billing** again. Without this step the **Billing** button (7.7 below) does not appear. This step can also be triggered for all billing units sharing the same settlement period via the **Ready for Billing** button on the property form.

3.7.7 7.7 Billing

Click **Billing** to create invoices and credit notes from the settlement results. For each contract with a settlement result the system creates:

- A credit note line reversing the advance payments booked during the year
- An invoice line for the actual costs allocated to the contract
- Optionally a journal entry for the owner allocation (WEG billing)

All postings of one billing run share a `billing_run_id` (YYYYMMDD–HHMMSS–U<user id>) for traceability.

If the property has **Collective billing** enabled, the *Billing* button is not shown on individual billing units; instead use the **Billing Property** button on the property form to trigger all billing units at once.

3.8 Typical Annual Workflow Summary

#	Action	Where
1	Create / update properties and objects	<i>Real Estate</i> ▢ <i>Properties</i>
2	Enter measurements (area, rooms)	Object form ▢ Measurements tab
3	Enter meter readings (initial, periodic)	<i>Real Estate</i> ▢ <i>Master Data</i> ▢ <i>Meter Readings</i>
4	Create / update contracts and terms	<i>Real Estate</i> ▢ <i>Contracts</i>
5	Generate invoices monthly (CreateContract-Moves)	<i>Real Estate</i> ▢ <i>Create Contract Moves</i>
6	Post invoices in accounting	<i>Accounting</i> ▢ <i>Invoices</i>
7	Create billing unit for the settlement year	<i>Real Estate</i> ▢ <i>Operation Costs</i> ▢ <i>Billing Units</i>
8	Add settlement units (cost types + allocation rules)	Billing unit form ▢ Settlement Units tab
9	Run Selection ▢ Compute Value Shares	Billing unit form (buttons)
10	Enter actual costs on settlement units	Settlement unit form
11	Compute Settlement Result	Billing unit form (button)
12	Click Ready for Billing	Billing unit form ▢ button / property form
13	Run Billing ▢ post invoices / credit notes	Billing unit form ▢ button / property form

DESIGN

This page describes the key architectural decisions, data model relationships, and invariants of the `real_estate` module. It is aimed at developers who want to extend or maintain the module.

4.1 Object Hierarchy

All real estate assets are represented by a single self-referential model `real_estate.base_object` using Tryton's `tree()` mixin. A single table stores all object types; the `type` field (`property`, `building`, `land`, `object`, `equipment`) distinguishes them.

Allowed parent-child combinations:

Object type	Valid parent types
<code>property</code>	— (root, no parent)
<code>building</code>	<code>property</code> , <code>building</code>
<code>land</code>	<code>property</code>
<code>object</code>	<code>property</code> , <code>building</code> , <code>land</code>
<code>equipment</code>	<code>building</code> , <code>land</code> , <code>object</code> , <code>equipment</code>

The `property` back-reference (stored `Many2One`, auto-computed from the `path`) points to the root of the tree. It is used extensively as a filter in contract items, billing units, and occupancy records.

The `path` field (Tryton `tree()` mixin) stores the full ancestor chain as a slash-separated string of IDs. SQL index on `path` enables fast subtree queries without recursive CTEs.

4.1.1 Single-table design rationale

Using one table rather than separate tables per type simplifies cross-type queries (e.g. “all objects under a `property`”), reduces join depth, and keeps the view/form layer uniform. Type-specific fields (meter fields, rental-object fields) are simply hidden by PYSON `states.invisible` conditions on the form view; they are `NULL` in the database for inapplicable types.

4.2 Measurement Group Hierarchy

`real_estate.measurement.type` supports an optional parent-group relationship: a type with `is_group = True` acts as a container for child types. Groups can be nested arbitrarily (multi-level hierarchies, e.g. *Usable Space* $\square$ *Living Space* + *Commercial Space*).

Two helper methods encapsulate hierarchy traversal:

get_hierarchy_ids() (instance method)

Returns [self.id] + all descendant IDs recursively, including intermediate group nodes. Used in ContractTerm.get_term_measurements to find all measurements relevant to a group m_type.

get_effective_ids(cls, m_type) (classmethod)

Returns only **leaf** (non-group) IDs, recursively expanded. Used in ContractTerm._sum_measurements and SettlementUnit._compute_value_shares so that allocation queries hit only concrete measurement records, not group containers.

Constraints enforced by validate_fields:

- A type may not reference itself or any of its descendants as parent (cycle detection via ancestor walk).
- All children must share the same UOM as their parent group.
- Only leaf types (is_group = False) may be assigned to individual real_estate.measurement records on objects.

4.3 Contract Architecture

A contract is assembled from three layers:

Contract (real_estate.contract)	
└─ ContractItem (real_estate.contract.item)	– "what is rented"
└─ ContractItemObject	– rental objects
└─ ContractTerm (real_estate.contract.term)	– "what is charged"
└─ ContractTermTax	– taxes
└─ ContractTermCashFlow	– cash flow entries

ContractItem links one or more BaseObject records of type object to the contract for a given validity period. The object selector is domain-restricted to the same property as the contract (property = Eval('property')), preventing cross-property assignments.

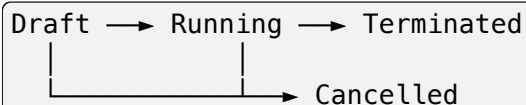
ContractTerm defines a recurring charge. A term always references a ContractItem(reference_item) so that measurement-based quantity derivation and operating cost allocation can trace back to the physical object.

ContractTermCashFlow is the projection / ledger entry:

- state = 'draft' — planned, no invoice yet
- state = 'done' — linked to an account.invoice.line

Cash flow entries carry create_moves_run_id (YYYYMMDD–HHMMSS–U<uid>) so every posting can be traced to the exact wizard run that created it.

4.4 Contract Workflow



Transitions:

- **Draft** → **Running** (button *Running*): activates the contract; sets running_by; triggers occupancy refresh.

- **Running** $\square$ **Terminated** (wizard *Terminate Contract*): records `termination_date`, `receipt_of_termination_notice`, `terminated_by`; triggers occupancy refresh.
- **Terminated** $\square$ **Running**: re-activates a contract (e.g. after a termination was withdrawn).
- **Draft / Running** $\square$ **Cancelled** (button *Cancel*): shows `ContractCancelWarning` if any done cash flow entries exist; deletes draft cash flow entries; triggers occupancy refresh. Cancelled contracts are excluded from all occupancy calculations.

4.5 Occupancy Tracking

`real_estate.base_object.occupancy` is a derived, read-only table that records the rental state of each object-type asset over time.

It is **not** maintained manually. Instead it is recomputed automatically whenever:

- A `ContractItem` or `ContractItemObject` is created, modified, or deleted (create / write / delete hooks).
- A contract changes state (via `Contract.write()` which checks `_COMPUTE_VALUE_SHARES_FIELDS` — this set includes 'state').

The recomputation filters contracts by state in ('draft', 'running', 'terminated'); cancelled contracts are excluded automatically.

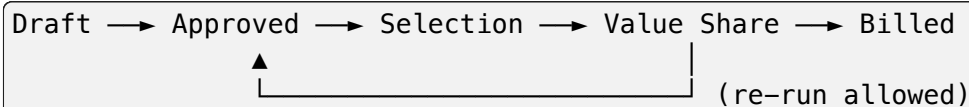
Occupancy states:

- **rented** — contract running, object occupied
- **under_negotiation** — contract draft, object provisionally reserved
- **vacant** — no active contract for the object

Overlap validation in `ContractItem._check_occupancy_overlap` rejects a second occupancy contract for the same object in the same period (hard error for `occupancy = True` contract types).

4.6 Operating Cost Settlement Pipeline

The settlement follows a strict state machine on `real_estate.billing_unit`:



Each stage builds on the previous:

1. **Approved** — billing unit is locked for editing; settlement units can be added.
2. **Selection** — `_run_selection()` identifies which objects and contracts fall inside the billing period. Creates one `CostShare` per (settlement_unit $\times$ object_or_contract) combination.
3. **Value Share** — `_compute_value_shares()` fills `CostShare.value_share` according to the allocation rule (measurement, consumption, per-unit, external, none). Errors are written to `CostShare.error_message`; the billing unit sub-state reflects whether errors exist.
4. **Billed** — `billing()` creates invoices and credit notes from `SettlementResult` records; writes `billing_run_id`.

Re-running *Selection* from *Value Share* deletes existing settlement results after a user confirmation warning (*SelectionWarning*).

Re-running *Compute Value Shares* automatically resets cost shares in state error to selection before recalculating, so corrected readings or measurements take effect without manual cleanup.

4.7 Allocation Rules

`SettlementUnit.allocation_rule` determines how costs are split:

allocation_by_measurement

Time-weighted share of a measurement value.

$$\text{value_share} = \text{measurement_value} \times \text{time_share} / \text{time_total}$$

The `m_type` field can reference a **group** measurement type; the system expands it to all leaf descendants via `get_effective_ids()`. A unit-fallback applies for non-group types: if the exact type is not found on an object, the system falls back to any measurement with the same UOM.

allocation_by_consumption

Raw meter reading consumption (*HeizkostenV*).

$$\text{value_share} = \text{end_reading} - \text{start_reading}$$

Readings are looked up within a tolerance window (`reading_pre_days` / `reading_post_days` on the cost type). If no reading is found, the cost share transitions to error.

Vacancy fallback: when the immediately following contract cost share has no start reading, the system uses the previous vacancy's start reading as a substitute, allowing a single read-out at move-out to serve both periods.

allocation_per_rental_unit

Pure time share:
$$\text{value_share} = \text{time_share} / \text{time_total}.$$

allocation_from_external_billing

No calculation. Amounts are entered manually on the cost share.

no_allocation

Entire cost remains unallocated.

4.8 Invoice Integration

The module extends three core accounting models:

account.invoice

Adds contract (Many2One to `real_estate.contract`) so every invoice is traceable to its origin contract.

account.invoice.line

Adds term (Many2One to `real_estate.contract.term`) and settlement_unit (Many2One to `real_estate.settlement_unit`). Term lines are created by `CreateContractMovesWizard`; settlement lines are created by `BillingUnit.billing()`.

account.move.line

Extended list view to expose contract and term context for journal entries.

`AccountContract` / `GeneralLedgerAccountContract` provide a contract-filtered view of the general ledger, accessible from the contract's *Account* tab.

4.9 Tryton Framework Patterns Used

Workflow

Used by BaseObject, Contract, and BillingUnit. State transitions are declared in `__setup__` via `cls._transitions` and enforced by `@Workflow.transition` decorators on buttons.

DeactivableMixin

Soft-delete for most master data and transactional records. The active flag is True by default; filtering in list views excludes inactive records automatically.

tree()

Self-referential parent-child for BaseObject and MeasurementType. Provides path field and sub-tree search helpers.

sequence_ordered() / re_sequence_ordered()

Integer sequence field with auto-increment helpers; used for consistent ordering in all One2Many tabs.

TaxableMixin

Tax calculation on Contract; used to derive tax amounts for invoice line generation.

Quantitative (custom fields.Numeric subclass)

Carries a unit reference and quantitative=True flag so the SAO client renders the associated UOM symbol next to the numeric value. Used for meter reading values and contract term quantities.

Cache

MeasurementType._get_default_type_cache and _get_window_domains_cache are invalidated in `on_modification` whenever a measurement type record changes.

UserWarning

ContractCancelWarning (cancel with existing postings), SelectionWarning (re-run selection), and OccupancyOverlapWarning follow the standard Tryton pattern: `Warning.format(key, records) + Warning.check(key) + raise`.

@set_employee

Decorator used by workflow transitions to stamp `running_by`, `terminated_by`, and `cancelled_by` from the current user.

API REFERENCE

This page lists every model provided by the `real_estate` module together with its key fields, states, and notable methods. Extension fields added to Tryton core models are listed separately at the bottom.

Notation: **[R]** = readonly, **[F]** = Function field, **[R/O]** = required on save.

5.1 Property Management

5.1.1 `real_estate.use_class`

Configurable use class for rental objects (Apartment, Retail, Parking, ...).

Field	Type	Description
<code>name</code>	Char [R/O]	Display name (translatable)
<code>sequence</code>	Integer [R/O]	Sort order; also used by demo scripts to find records
<code>has_basement_nr</code>	Boolean	Show <i>Basement Number</i> field on objects of this class
<code>has_parking_nr</code>	Boolean	Show <i>Parking Number</i> field on objects of this class

5.1.2 `real_estate.address`

Structured address with granular sub-fields.

Field	Type	Description
<code>street_name</code>	Char	Street name without number
<code>building_number</code>	Char	House / building number
<code>unit_number</code>	Char	Apartment / unit identifier
<code>floor_number</code>	Char	Floor
<code>room_number</code>	Char	Room identifier
<code>post_box</code>	Char	Post box number
<code>postal_code</code>	Char	Postal code
<code>city</code>	Char	City
<code>country</code>	Many2One	<code>country.country</code>
<code>street</code>	Char [F]	Composed <code>street_name</code> + <code>building_number</code> for display

5.1.3 real_estate.base_object

Central entity for every real estate asset.

Types: property · building · land · object · equipment

Workflow: Draft ☐ Approved ☐ Locked (+ Deactivated)

Field	Type	Description
name	Char [R/O]	Description / display name
type	Selection [R/O]	Object type; controls visible fields
type_of_use	Selection	residential / commercial / property / internal
use_class	Many2One	real_estate.use_class; visible for type <i>object</i> only
company	Many2One [R/O]	company.company
start_date	Date [R/O]	Date the asset was commissioned
end_date	Date	Date the asset was decommissioned
address	Many2One	real_estate.address
property	Many2One [R] [F]	Root property object (auto-derived from tree path)
parent	Many2One	Direct parent in the object tree
children	One2Many	Direct children
path	Char [R]	Full ancestor path (tree mixin)
state	Selection	draft / approved / locked / deactivated
object_number	Char [R]	Auto-generated object identifier
measurements	One2Many	real_estate.measurement records
parties	One2Many	real_estate.object_party (roles)
billing_units	One2Many	real_estate.billing_unit (property only)
occupancy	One2Many [R]	real_estate.base_object.occupancy (computed)
billing_as	Selection	single / collective — aggregation for cost billing
collective_billing	Boolean [F]	Derived from property's billing_as
year_of_construction	Char	4-digit year (building only)
number_of_floors	Integer	Number of floors (building only)
floor	Integer	Floor number (object only)
basement_nr	Char	Basement storage number (object, use_class dependent)
parking_nr	Char	Parking number (object, use_class dependent)
e_type	Selection	Equipment sub-type: meters / other (equipment only)
meter_unit	Many2One	UOM for meter readings (equipment/meters only)
meter_is_counter	Boolean	True for cumulative counters (consumption = diff to prev. reading)
meter_factor	Float	Conversion factor applied to raw readings (default 1.0)
meter_no_decimals	Boolean	Readings must be integer values
meter_id	Char [F]	ID of the most recent reading
meter_last_value	Quantitative [F]	Last recorded reading value
meter_last_consumption	Quantitative [F]	Consumption since the previous reading

5.1.4 `real_estate.base_object.occupancy`

Read-only occupancy ledger; recomputed automatically on contract changes.

Field	Type	Description
<code>base_object</code>	Many2One [R]	The object being tracked
<code>property</code>	Many2One [R]	Root property (denormalized)
<code>start_date</code>	Date [R]	First day of occupancy period
<code>end_date</code>	Date [R]	Last day (None = open-ended)
<code>state</code>	Selection [R]	<code>rented / vacant / under_negotiation</code>
<code>contract</code>	Many2One [R]	Source contract (None for vacancy periods)

5.1.5 `real_estate.measurement.type`

Named measurement type linked to a UOM. Supports group hierarchy.

Field	Type	Description
<code>name</code>	Char [R/O]	Display name (translatable)
<code>unit</code>	Many2One [R/O]	<code>product.uom</code>
<code>types</code>	MultiSelection	Object types this measurement applies to
<code>is_group</code>	Boolean	Marks this type as a group container
<code>parent</code>	Many2One	Parent group (<code>is_group = True</code> required on parent)
<code>children</code>	One2Many	Child types (visible when <code>is_group = True</code>)
<code>default</code>	Boolean	Use as default type for the object type
<code>no_print</code>	Boolean	Exclude from printed reports

Key methods:

`get_hierarchy_ids()`

Returns `self.id` plus all descendant IDs recursively (including groups).

`get_effective_ids(cls, m_type)`

Returns only leaf (non-group) IDs, recursively expanded from `m_type`.

5.1.6 `real_estate.measurement`

Associates a numeric value with a `BaseObject` for a given date.

Field	Type	Description
<code>base_object</code>	Many2One [R/O]	The object being measured
<code>m_type</code>	Many2One [R/O]	<code>real_estate.measurement.type</code> (leaf types only)
<code>valid_from</code>	Date [R/O]	Date from which this measurement applies
<code>value</code>	Float [R/O]	Measured value
<code>symbol</code>	Char [F]	UOM symbol (derived from <code>m_type</code>)
<code>no_print</code>	Boolean	Exclude from printed reports

5.1.7 `real_estate.meter_reading`

Meter reading linked to an equipment object of type *meters*.

Field	Type	Description
<code>base_object</code>	Many2One [R/O]	Equipment object (<code>e_type = 'meters'</code>)
<code>meter_id</code>	Char [R/O]	Physical meter identifier
<code>reading_date</code>	Date [R/O]	Date of reading
<code>m_type</code>	Selection [R/O]	<code>initial / reading / estimate / final</code>
<code>value</code>	Quantitative [R/O]	Raw meter value
<code>unit</code>	Many2One [F]	UOM (derived from equipment's <code>meter_unit</code>)
<code>consumption</code>	Quantitative [F]	Value minus previous reading (counter meters only)
<code>reading_user</code>	Many2One	User who recorded the reading
<code>comment</code>	Char	Free text

5.1.8 `real_estate.object_party.role`

Configurable role definition for object parties.

Fields: `name` (Char), `sequence` (Integer), `types` (MultiSelection of object types), `default` (Boolean).

5.1.9 `real_estate.object_party`

Links a `party.party` to a `BaseObject` with a typed role.

Fields: `base_object` (Many2One), `party` (Many2One), `role` (Many2One $\square$ `object_party.role`), `valid_from` (Date), `valid_to` (Date).

5.2 Contract Management

5.2.1 real_estate.contract.type

Template controlling contract behaviour.

Field	Type	Description
name	Char [R/O]	Display name (translatable)
sequence	Integer [R/O]	Sort order
types_of_use	MultiSelection	Which type_of_use values this type applies to
invoice_type	Selection	out (debit/receivable) or in (credit/payable)
account	Many2One	Default party account for invoice lines
taxes	Many2Many	Default taxes applied to all terms
account_journal	Many2One	Default posting journal
prefix	Char [R/O]	Contract number prefix (e.g. 1)
start_number	Integer [R/O]	First contract number
step_item	Integer [R/O]	Sequence step for contract items
step_term	Integer [R/O]	Sequence step for contract terms
occupancy	Boolean	Enforce occupancy exclusivity for this contract type
mark	Char	Periodic posting mark (Debit / Credit)

5.2.2 real_estate.contract.term.type

Template for a recurring charge line.

Field	Type	Description
name	Char [R/O]	Display name (translatable)
sequence	Integer [R/O]	Sort order; also used by scripts to locate records
types_of_use	MultiSelection	Applicable type-of-use values
m_type	Many2One	Measurement type for default quantity derivation
default_quantity	Float	Fallback quantity when no measurement is found
account	Many2One	Default account for invoice lines
rhythm	Integer	Number of rhythm units per invoice
rhythm_type	Selection	daily / weekly / monthly / quarterly / annually / one_time
rhythm_start	Selection	Day of month (1–28) when the first invoice of the period is due

5.2.3 real_estate.contract

Main contract record.

Workflow: Draft ☐ Running ☐ Terminated / Cancelled

Field	Type	Description
c_type	Many2One [R/O]	real_estate.contract.type
property	Many2One [R/O]	Root property
contractual_partner	Many2One [R/O]	party.party (tenant / owner)
invoice_address	Many2One	party.address used on invoices
type_of_use	Selection	residential / commercial / property / internal
currency	Many2One [R/O]	Billing currency
start_date	Date [R/O]	Contract start
end_date	Date [F]	Effective end (max of item valid_to dates)
state	Selection	draft / running / terminated / cancelled
items	One2Many	real_estate.contract.item
terms	One2Many	real_estate.contract.term
logs	One2Many [R]	real_estate.contract.log
running_by	Many2One [R]	User who activated the contract
terminated_by	Many2One [R]	User who terminated the contract
cancelled_by	Many2One [R]	User who cancelled the contract
termination_date	Date	Last day of the tenancy
receipt_of_termination	Date	Date the termination notice was received
termination_reason	Text	Free-text reason

5.2.4 real_estate.contract.item

Assigns rental objects to a contract for a validity period.

Field	Type	Description
contract	Many2One [R/O]	Parent contract
label	Char	Optional label (overrides auto-name)
valid_from	Date [R/O]	Start of object assignment
valid_to	Date	End of object assignment (open if None)
objects	One2Many	real_estate.contract.item.object (rental objects)
property	Many2One [F]	Derived from contract.property

5.2.5 `real_estate.contract.item.object`

Junction between a `ContractItem` and a `BaseObject`.

Field	Type	Description
<code>item</code>	Many2One [R/O]	Parent <code>ContractItem</code>
<code>object</code>	Many2One [R/O]	<code>real_estate.base_object</code> (type <i>object</i>); domain-restricted to objects of the same property as the contract
<code>property</code>	Many2One [F]	Derived from <code>item.contract.property</code>

5.2.6 `real_estate.contract.term`

Recurring charge line on a contract.

Field	Type	Description
<code>contract</code>	Many2One [R/O]	Parent contract
<code>term_type</code>	Many2One [R/O]	<code>real_estate.contract.term.type</code>
<code>reference_item</code>	Many2One	<code>ContractItem</code> this charge relates to
<code>valid_from</code>	Date [R/O]	Start of charge period
<code>valid_to</code>	Date	End of charge period (open if None)
<code>rhythm</code>	Integer	Billing frequency count
<code>rhythm_type</code>	Selection	daily / weekly / monthly / quarterly / annually / one_time
<code>rhythm_start</code>	Selection	Day-of-month start (1–28)
<code>quantity</code>	Quantitative	Units per billing period
<code>unit</code>	Many2One [F]	UOM (from <code>term_type.m_type.unit</code>)
<code>unit_price</code>	Monetary	Price per unit per period
<code>taxes</code>	Many2Many	Applied taxes
<code>cash_flows</code>	One2Many	<code>real_estate.contract.term.cash_flow</code>

5.2.7 `real_estate.contract.term.cash_flow`

Single projected or realised cash flow entry.

Field	Type	Description
term	Many2One [R/O]	Parent ContractTerm
state	Selection	draft (planned) / done (invoiced)
posting_date	Date	Invoice / posting date
document_date	Date	Document date on the invoice
due_date	Date	Payment due date
invoice_line	Many2One [R]	account.invoice.line (set when state = done)
create_moves_run_id	Char [R]	Run ID of the CreateContractMoves wizard invocation
invoice_state	Selection [F]	Derived from linked invoice: draft / validated / posted / paid

5.2.8 real_estate.contract.log

Append-only audit log entry on a contract.

Fields: contract (Many2One), event (Char), description (Text), create_date (DateTime [R]), create_uid (Many2One [R]).

5.3 Operating Cost Settlement

5.3.1 real_estate.cost_category_group

Groups cost types for reporting.

Fields: name (Char [R/O]), sequence (Integer [R/O]).

5.3.2 real_estate.cost_type

Individual operating cost item (e.g. *Grundsteuer*).

Field	Type	Description
name	Char [R/O]	Display name (translatable)
sequence	Integer [R/O]	Sort order; used by scripts to locate records
category_group	Many2One	real_estate.cost_category_group
comment	Text	BetrKV paragraph reference or free text
no_print	Boolean	Exclude from settlement printout
reading_pre_days	Integer	Days before target date within which a meter reading is accepted (default 7; allocation_by_consumption only)
reading_post_days	Integer	Days after target date within which a meter reading is accepted (default 7; allocation_by_consumption only)

5.3.3 real_estate.billing_unit

Annual (or period) operating cost settlement for one property.

Workflow: Draft ☐ Approved ☐ Selection ☐ Value Share ☐ Billed

Field	Type	Description
property	Many2One [R/O]	Root property
description	Char [R/O]	Free-text label (e.g. "Kalte Betriebskosten 2025")
start_date	Date [R/O]	First day of settlement period
end_date	Date [F]	Last day (defaults to end of calendar year of start_date)
calculation_method	Selection	rental_apartment (§§ 1–2 BetrKV) / WEG_billing
billing_type	Selection	planned_billing / actual_billing
state	Selection	draft / approved / selection / value_share / billed
sub_state	Selection [F]	ok / error / warning (derived from cost share states)
term_types_of_use	MultiSelection	Term type sequences counted as advance payments
settlement_units	One2Many	real_estate.settlement_unit
settlement_results	One2Many	real_estate.settlement_result
moves	One2Many	real_estate.billing_unit.moves
billing_run_id	Char [R]	YYYYMMDD–HHMMSS–U<uid> stamped at end of billing()
sum_planned_costs	Monetary [F]	Sum of planned costs across all settlement units
sum_actual_costs	Monetary [F]	Sum of actual costs
sum_advanced_paymer	Monetary [F]	Sum of posted/paid advance payments
sum_refund_receival	Monetary [F]	sum_actual_costs – sum_advanced_payment

5.3.4 real_estate.settlement_unit

One cost-type line within a billing unit.

Field	Type	Description
billing_unit	Many2One [R/O]	Parent billing unit
type	Many2One [R/O]	real_estate.cost_type
allocation_rule	Selection [R/O]	allocation_by_measurement / allocation_by_consumption / allocation_per_rental_unit / allocation_from_external_billing / no_allocation
vacancy	Selection	by_owner / unallocated
m_type	Many2One	Measurement type for measurement-based allocation
meter_unit	Many2One	UOM for consumption-based allocation
reg_ex_object	Char	Regex to filter rental objects (applied to base_object.name)
reg_ex_meter	Char	Regex to filter meter equipment (applied to base_object.name)
value_total	Float	Total allocation base (sum of all cost shares' value_share)
cost_shares	One2Many	real_estate.cost_share

5.3.5 real_estate.cost_share

Intermediate allocation record: share of one cost type for one contract/object within a billing period.

States: preparation ☐ selection ☐ estimated_value_share ☐ value_share · error

Field	Type	Description
settlement_unit	Many2One [R/O]	Parent settlement unit
contract	Many2One	Contract occupying the object (None = vacancy)
base_object	Many2One	The rental object
start_date	Date	Start of this share's occupancy period
end_date	Date	End of this share's occupancy period
state	Selection	See states above
value_share	Float	Allocation factor (measurement value × time fraction, or raw consumption, or time fraction)
time_share	Integer [F]	Days within the billing period
allocation_rule	Selection [F]	Mirrored from the parent settlement unit
external_billing	Boolean [F]	Mirrored from the parent settlement unit's billing unit
planned_costs	Monetary	Budgeted cost for this share. Readonly unless external_billing is set (otherwise computed, not entered manually)
actual_costs	Monetary	Real cost for this share. Readonly unless external_billing is set (otherwise computed, not entered manually)
error_message	Char [R]	Human-readable error (set when state = error)

5.3.6 real_estate.settlement_result

Final per-contract settlement for one billing unit.

States: approved ☐ billed

Field	Type	Description
billing_unit	Many2One [R/O]	Parent billing unit
contract	Many2One	Tenant contract
base_object	Many2One	Rental object (denormalized)
start_date	Date	Settlement period start
end_date	Date	Settlement period end
state	Selection	approved / billed
planned_costs	Monetary	Sum of planned costs from cost shares
actual_costs	Monetary	Sum of actual costs from cost shares
advanced_payment	Monetary	Sum of posted/paid advance invoice lines
refund_receivable	Monetary [F]	actual_costs – advanced_payment (positive = refund)
term	Many2One	The single advance-payment term (if unambiguous)
invoice	Many2One [R]	Settlement invoice created by billing()

5.3.7 real_estate.billing_unit.moves

One record per posting created by `billing()`.

Field	Type	Description
<code>billing_unit</code>	Many2One [R/O]	Parent billing unit
<code>settlement_result</code>	Many2One	Source settlement result
<code>contract</code>	Many2One	Source contract
<code>billing_run_id</code>	Char [R]	Same value as parent <code>BillingUnit.billing_run_id</code>
<code>moves_advanced_paym</code>	Many2One	Credit-note line reversing advance payments
<code>moves_actual_costs</code>	Many2One	Invoice line for actual costs
<code>moves_alloc_by_owne</code>	Many2One	Journal move line for owner allocation (WEG only)

5.4 Extensions to Core Modules

party.party (party.py)

Adds sequence (Integer, sort key) and salutation (Char).

account.invoice (invoice.py)

Adds contract (Many2One → `real_estate.contract`).

account.invoice.line (invoice.py)

Adds real-estate assignment fields, gated by `assignment_control` (Selection: *All*, *contract*, *operating_costs*, *settlement_result_contract*, *settlement_result_vacant*): *contract / term* (originating contract / term), *base_object*, *billing_unit / settlement_unit* (for operating-cost lines), *property* [F], *service_period_from / service_period_to*, *estg_35a* (Selection, German §35a EStG tax-deduction category), and *invoice_date / tax_amount / total_amount* [F].

account.move.line (invoice.py, class AccountMoveLine)

Mirrors the same fields as `account.invoice.line` above (*assignment_control*, *contract*, *term*, *base_object*, *billing_unit*, *settlement_unit*, *property* [F]) directly on the journal entry line.

account.configuration (account_configuration.py)

Adds three real-estate defaults, all Many2One: *re_account_allocation_by_owner* (Vacancy Cost Account, used for direct GL postings of vacancy settlement results), *re_journal_billing* (journal for those postings), and *re_payment_term_billing* (default payment term for operating cost settlement invoices).

res.user (res.py)

Adds phone (Char) and mobile (Char).

5.5 Wizards

real_estate.contract.create_moves.wizard

Batch invoice generation. Start state *start* collects *create_date*, optional *property* and *contract* filters, and *action* (*create* / *re_calc* / *re_calc_and_create*).

real_estate.terminate_contract.wizard

Terminates a running contract. Collects *terminated_by_type*,

receipt_of_termination_notice, termination_reason, and termination_date (or computes it from a notice period in months).

real_estate.estimate_consumption.wizard

Two-step wizard on meter equipment objects. Step 1 collects per_date, meter_id, and reason; step 2 shows the simulated consumption and lets the user confirm or adjust the estimated_value before saving a new MeterReading.

RELEASE NOTES

MODULE DEPENDENCIES

ir, res, company, party, product, currency, country, account, account_invoice, account_deposit, account_payment_clearing, account_tax_non_deductible, bank

bank is used only by the BVED export's A-/M-Satz bank-field lookup (party.bank_accounts_used $\square$ IBAN $\square$ classic Kontonummer/Bankleitzahl) — see *BVED External Billing Interface* below.

INSTALLATION

```
pip install -e .          # development install
pip install -e '.[test]'  # with test dependencies
```

The module entry point is declared in `setup.py`:

```
[trytond.modules]
real_estate = trytond.modules.real_estate
```

Link or install the package inside the Tryton modules directory so that `trytond-admin -u real_estate` can find it.

CONFIGURATION

The following master data must be set up before the module can be used:

real_estate.object_party.role

Partner roles in combination with the property type (`base_object.type`), e.g. *Tenant*, *Owner*, *Administrator*.

real_estate.measurement.type

Named measurements linked to a unit of measure, e.g. *Living Area (m²)*, *Number of Rooms*, *Gross Floor Area (m²)*.

real_estate.contract.type

Contract types defining invoice direction (in/out), default journal, tax defaults, contract number prefix, and whether occupancy exclusivity is enforced.

real_estate.contract.term.type

Term type definitions with default rhythm (monthly / quarterly / ...), default quantity source (measurement type), and default account.

real_estate.use_class

Dynamic use-class catalogue replacing the former static selection field. Each record carries two boolean flags:

has_basement_nr

Show/hide the *Basement Number* field on rental objects.

has_parking_nr

Show/hide the *Parking Number* field on rental objects.

Six default records are loaded at module installation: *Apartment* (`has_basement_nr`), *Office* (`has_basement_nr`), *Retail* (`has_basement_nr`), *Warehouse* (`has_basement_nr`), *Parking* (`has_parking_nr`), *Garage* (`has_parking_nr`). Additional classes can be created without changing code.

real_estate.cost_category_group / real_estate.cost_type

Cost categories (e.g. *Heating*, *Water*) and individual cost types used to structure operating cost settlements. Default values for German BetrKV (§ 2) are loaded at module installation.

real_estate.re_accounting(re_accounting.py)

Standalone, company-scoped real-estate accounting configuration, referenced one-directionally from `company.company.re_accounting(company.py)`; optional — a company without a linked `re_accounting` record gets none of the automation below).

re_account_allocation_by_owner

Vacancy cost account (debit and credit side of vacancy postings).

re_journal_billing

Journal used for direct GL postings in operating cost settlements.

re_payment_term_billing

Default payment term for operating cost settlement invoices, used when the contract itself has none set.

co2_landlord_share_commercial

Default CO2 cost landlord share (%; 0–100) for commercial properties, which are not covered by the residential 10-tier distribution model — see *CO2 Cost Allocation (CO2KostAufG)* under *Operating Cost Settlement* below. Shown right before the *Cron Tasks* table on the form.

cron_tasks

One2Many to `real_estate.cron_task` — the company’s scheduled task configuration, see below.

real_estate.cron_task (cron_task.py)

Per-re_accounting (i.e. per-company) row configuring one recurring operation: task (selection, currently `update_contract_status` / `update_contract_cash_flow` / `book_contract_cash_flow` / `update_option_rate`), `valid_from`, `valid_until`, `interval_days`, `interval_months`, `schedule_day_of_month`, `horizon_months_ahead`, `invoice_state`, `future_contracts_horizon_days`, `last_run` (updated by the dispatcher), `active`. Task names are deliberately rhythm-agnostic (no “daily”/“rolling”/... in the name) — how often each task actually runs is a per-row configuration choice made by the user, not implied by the task itself; see *Scheduling and parameters* below. None of `interval_days`/`interval_months`/`schedule_day_of_month` is individually required, but `validate()` rejects a row that has none of the three set. A unique SQL constraint prevents duplicate rows for the same (`re_accounting`, `task`) pair. No rows are seeded automatically — a company gets no automatic behaviour until at least one row is added under the *Cron Tasks* tab of its `real_estate.re_accounting` record.

Dispatch. A single `ir.cron` entry (“Real Estate Daily Tasks”, method `real_estate.contract|cron_daily` — registered via a small `ir.cron.method` selection extension in `ir.py`, since that field is otherwise a fixed core list) runs daily and calls `Contract.cron_daily()` (`contract_core.py`). It iterates all active `cron_task` rows and, for each one that is due (see *Scheduling* below), calls the matching `Contract._cron_<task>(re_accounting, task)` classmethod and updates `last_run`. A future recurring operation only needs a new `_cron_<code>` classmethod plus a new `get_tasks()` option — no new `ir.cron` entry.

Scheduling and parameters. Each `cron_task` row carries its own parameters, read by its handler from the task record passed in, and its own rhythm - e.g. `update_contract_status` daily (`interval_days = 1`), `update_contract_cash_flow` every six months (`interval_months = 6`), `book_contract_cash_flow` and `update_option_rate` monthly (`schedule_day_of_month` set, e.g. 15 and 1 respectively).

`valid_from`/`valid_until` (both optional) are hard lower/upper bounds checked before all three modes below - the task never runs while `today < valid_from` or `today > valid_until`. If the task has never run yet and today is already past `valid_from`, it becomes due immediately (using today, not `valid_from`, as the actual first `last_run`). E.g. setting `valid_from = 15.03.2026` on a row activated on 20.03.2026 makes it run right away on 20.03.2026, not wait until the next scheduled date. `valid_until` just stops execution once passed - the row itself, and its `last_run` history, stay in place. `validate()` rejects a row where `valid_until` is before `valid_from`.

Three scheduling modes, checked by `Contract._cron_task_is_due()` in this priority order:

1. **Day-of-month-based:** if `schedule_day_of_month` (1-31) is set, it takes priority over both interval fields — the task runs at most once a month, on or after that calendar day (capped to the last day of shorter months), guarded by comparing `last_run`’s year/month to today’s so a delayed run still catches up without re-running twice in the same month.
2. **Month-interval-based:** otherwise, if `interval_months` is set, it takes priority over `interval_days`. If `valid_from` is also set, `Contract._add_months()` computes the ex-

act recurring date `valid_from + k * interval_months` months (same day-of-month as `valid_from`, clamped to shorter months), advancing `k` past `last_run` each time - so e.g. `valid_from = 15.03.2026` with `interval_months = 1` runs on 15.03., 15.04., 15.05., ... and re-aligns to the correct slot even after a delayed run, instead of drifting from whatever date the delayed run actually happened on. Without `valid_from`, falls back to a looser check: due once at least that many calendar months (year/month difference, not exact days) have passed since `last_run` - a convenience for longer rhythms (e.g. 6 for half-yearly) without day-of-month precision.

3. **Day-interval-based** (default): otherwise, due once today - `last_run >= interval_days` (or `last_run` is unset).

`_cron_update_contract_status`

Auto-terminates running contracts of the company whose `end_date` has passed and which have no active termination yet (`termination_date` unset): sets `state = 'terminated'`, `termination_date = end_date`, `terminated_by_type = 'expired'`, and a `contract.log` entry — reusing the existing terminated state rather than adding a new one, so all state-dependent logic (occupancy, `create_moves`, item validation) keeps working unchanged. See `get_effective_end_date()` (`termination_date` if set and earlier than `end_date`, else `end_date`) which already governs occupancy for both running and terminated contracts regardless of whether this task has run yet.

`_cron_update_contract_cash_flow`

Cash-flow recalculation only (`action='re_calc'`): calls `Contract.call_create_moves()` for running/terminated contracts of the company, and for not-yet-started contracts whose `start_date` is within today + `future_contracts_horizon_days` (the row's own field, default 60) — already-running/-ended contracts are always included regardless of this value; it only decides how far ahead of their actual start not-yet-started contracts get pulled in early. For every contract included, `Term.re_calc()` (`contract_term.py`) recalculates that term's cash flow up to a **fixed** horizon of exactly 1 year from today (`_re_calc_year = 1`, not related to and not affected by `future_contracts_horizon_days`). It does **not** create invoices — booking is a separate, explicitly scheduled step, see `_cron_book_contract_cash_flow` below.

`_cron_book_contract_cash_flow`

Books (creates invoices for) all due terms, using `action='re_calc_and_create'` so it is self-sufficient regardless of whether `_cron_update_contract_cash_flow`'s horizon already covers the target date. The horizon is the end of the month that is `horizon_months_ahead` months after the run date — e.g. with `schedule_day_of_month = 15` and `horizon_months_ahead = 1`, a run on 15.07 books everything due up to 31.08. The row's own `invoice_state` (draft/posted, default draft) is passed through to `call_create_moves/_create_moves` and determines whether the created invoices are posted immediately or left as drafts for manual review. Note: each invoice's own `invoice_date/` `accounting_date` is *not* forced to this month-end horizon — it is the individual term's own computed `document_date` (see `_create_moves`, `inv_date = invoice_date` or `document_date` with `invoice_date=None` here), which may differ per term/rhythm.

`_cron_update_option_rate`

Recomputes and, where the rate actually changed, books new option rates via `OptionRate.process_update()` (`option_rate.py` — the same logic as the manual `real_estate.option_rate_update.wizard`, see *Wizards* below) for every property of every company using this `re_accounting` configuration, as of today's date. Intended to run monthly with `schedule_day_of_month = 1`, which also determines the `effective_date` used by `process_update` (the 1st of the current month). Per-record results (created/updated/unchanged/ skipped counts, plus one detail line per processed object) go to the

Python logger, not contract . log, since this task is not tied to individual contracts.

ACCESS CONTROL

Seven user groups are provided. Access is additive — a user may belong to multiple groups.

group_real_estate_admin — Real Estate Administration

Full CRUD on all module models. Intended for system administrators and property managers with unrestricted access. The built-in admin user is automatically assigned to this group at installation.

group_real_estate_object — Real Estate Object

Full CRUD on property and object data (`base_object`, `address`, `measurement`, `meter_reading`, `object_party`, `occupancy`). Read-only access to contract, billing, and configuration models. Intended for facility managers who maintain the property master data but do not manage contracts or run settlements.

Exception: also has full CRUD on `real_estate.option_rate` (in addition to `group_real_estate_admin`), since option rates are maintained together with the property/object master data via the *Update Option Rates* wizard — see *Option Rate* below. The `group_real_estate_contract` and `group_real_estate_billing` groups remain read-only on it.

group_real_estate_contract — Real Estate Contract

Full CRUD on contract data (`contract`, `contract.term`, `contract.item`, `contract.log`, `contract.term.cash_flow`, `contract.term.tax`, `account_contract`). Read-only access to property, billing, and configuration models. Intended for property managers / letting agents.

Exception: also has full CRUD on `real_estate.contract.term.adjustment` (in addition to `group_real_estate_admin`) — see *Contract Term Adjustment* below. The `group_real_estate_object` and `group_real_estate_billing` groups remain read-only on it.

group_real_estate_billing — Real Estate Billing

Full CRUD on all operating cost and settlement models (`billing_unit`, `billing_unit.log`, `billing_unit.moves`, `settlement_unit`, `settlement_result`, `cost_share`, `cost_category_group`, `cost_type`, `account.configuration.real_estate`), and on the BVED external billing models (`bved.service_provider`, `bved.provider_assignment`, `bved.object_number`, `bved.object_number.m_satz_line`, `bved.export`, `bved.import`, `bved.import.line` — see *BVED External Billing Interface* below). Read-only access to property and contract models. Intended for the operating cost accountant who runs the annual settlement but does not modify contracts or property master data.

Exception: the `group_real_estate_contract` group also has read-only access to all six BVED models (shared master/exchange data, not contract-specific); `group_real_estate_object` has none.

group_real_estate_view — Real Estate View (German: *Immobilien Anzeigen*)

Read-only (`perm_read=True`, `perm_write/perm_create/perm_delete=False`) on every

module model that has explicit `ir.model.access` records — i.e. every model listed in the permission matrix below, without exception. Defined in its own file, `group_real_estate_view.xml` (one `res.group` record plus one `ir.model.access` record per model, generated by enumerating every `<record model="ir.model.access">` across the module's XML files via an XML parser — not a plain-text/regex search, which would also match commented-out `ir.model.access` blocks such as the disabled `real_estate.address.format` one in `address.xml`). Intended for users who need full visibility across the module (e.g. auditors, read-only reporting accounts) without being able to change anything, independent of which of the other four functional groups they would otherwise need for write access. Deliberately scoped to the module's own `real_estate.*` models only — it does not touch access to core models the module merely extends with extra fields (`party.party`, `account.invoice`, `account.move.line`, `company.company`), since those already have their own, separate access control.

group_account_view — Accounting View (German: *Rechnungswesen Anzeigen*)

Read-only counterpart to core's own `account.group_account` ("Accounting" / German *Rechnungswesen*) group — same ten models (`account.aged_balance`, `account.general_ledger`, `account`, `account.general_ledger.account.party`, `account.invoice`, `account.invoice.line`, `account.invoice.payment.mean.rule`, `account.journal.period`, `account.move`, `account.move.reconciliation`, `account.move.template`) that `group_account` itself has explicit `ir.model.access` records for (found by querying a live database's `ir.model.access` table for that group — `group_account`'s own permissions are additive with its child `group_account_admin`'s via `res.group.parent`, so `account.account/account.period/account.fiscalyear/account.tax` etc. are only reachable through that child group, not directly on `group_account`, and are intentionally excluded here). All ten downgraded to read-only. Defined in `group_account_view.xml` — kept as a module of its own file, not merged into `group_real_estate_view.xml`, since it grants access on core accounting models rather than this module's own; added here (rather than patched into core's `account` module, which this module must not modify) because `real_estate` already depends on `account`. Independent of `group_real_estate_view` — a user typically needs both to see the full picture (real-estate objects/contracts plus the accounting moves/invoices behind them) without being able to change either.

Note

`ir.model.access` alone does not make a menu item appear in the client. The top-level *Rechnungswesen* menu (`account.menu_account`) is only shown to groups explicitly linked to it via `ir.ui.menu-res.group` — by default just `group_account` and `group_account_admin` (this is the *only* menu in the whole `account` module gated this way, confirmed by querying a live database for every menu linked to `group_account`; everything below it in the tree is unrestricted at the menu level, relying solely on each item's own `ir.model.access` once opened). Without an equivalent link, a user placed in `group_account_view` alone would have read access to all ten models yet never see the menu to reach them — `group_account_view.xml` therefore also adds `menu_account_group_account_view` (an `ir.ui.menu-res.group` record pointing `group_account_view` at `account.menu_account`), the same mechanism core itself uses for the other two groups.

group_party_access — Party Access (German: *Partner Zugriff*)

`party.party` has no `ir.model.access` records anywhere in core or in the rest of this module — Tryton's default for a model with zero access rows is full CRUD for everyone, since parties are normally shared openly across whatever modules happen to be installed alongside `real_estate` (sales, purchasing, accounting...). Per explicit request, party editing is now a separately-assigned privilege: only members of `group_party_access` may create, modify, or delete parties; every *other* role — including this module's own `group_real_estate_admin/object/contract/billing`

and both “view” groups above — is downgraded to read-only (still needed so party names keep displaying wherever a party is merely referenced: contracts, invoices, lists...). Defined in `group_party_access.xml`: a `_default` (no-group) `ir.model.access` row (`perm_read=True`, the rest `False`) plus one full-CRUD row for `group_party_access` itself. The built-in admin user is auto-assigned to this group at installation (`res.user-res.group`), matching how core’s own `group_account/group_account_admin` and this module’s own `group_real_estate_admin` already do, so the out-of-the-box admin account is not locked out.

Warning

Deliberately does **not** reference any group from optional modules this one does not depend on (sale, purchase, ...) — doing so from XML would break installation wherever those modules are absent. If such a module’s own users need to keep creating or editing parties after this change, an administrator must add them to `group_party_access` directly (Administration  User  the user’s *Groups* field), independently of this module.

Permission matrix (C = CRUD · R = read · – = no access). The view group is not shown as its own column since its value is R for every single row without exception:

Model	admin	object	contract	billing
<code>real_estate.address</code>	C	C	R	R
<code>real_estate.base_object</code>	C	C	R	R
<code>real_estate.base_object.occupancy</code>	C	C	C	R
<code>real_estate.option_rate</code>	C	C	R	R
<code>real_estate.meter_reading</code>	C	C	R	R
<code>real_estate.measurement.type</code>	C	R	R	R
<code>real_estate.use_class</code>	C	R	R	R
<code>real_estate.measurement</code>	C	C	R	R
<code>real_estate.object_party</code>	C	C	R	R
<code>real_estate.object_party.role</code>	C	R	R	R
<code>real_estate.contract</code>	C	R	C	R
<code>real_estate.contract.item</code>	C	R	C	R
<code>real_estate.contract.term</code>	C	R	C	R
<code>real_estate.contract.term.tax</code>	C	R	C	R
<code>real_estate.contract.log</code>	C	R	C	R
<code>real_estate.contract.account_contract</code>	C	R	C	R
<code>real_estate.contract.term.cash_flow</code>	C	R	C	R
<code>real_estate.contract.term.adjustment</code>	C	R	C	R
<code>real_estate.account_contract</code>	C	R	R	R
<code>real_estate.contract.type</code>	C	R	R	R
<code>real_estate.contract.term.type</code>	C	R	R	R
<code>real_estate.re_accounting</code>	C	R	R	R
<code>real_estate.cron_task</code>	C	R	R	R
<code>real_estate.co2_emission_share</code>	C	R	R	R
<code>real_estate.contract.type.tax</code>	C	—	R	R
<code>real_estate.cost_category_group</code>	C	R	R	C
<code>real_estate.cost_type</code>	C	R	R	C
<code>real_estate.co2_kostaufg</code>	C	R	R	C
<code>real_estate.co2_kostaufg.consumption</code>	C	R	R	C
<code>real_estate.billing_unit</code>	C	R	R	C
<code>real_estate.billing_unit.log</code>	C	R	R	C

continues on next page

Table 1 – continued from previous page

Model	admin	object	contract	billing
real_estate.billing_unit.moves	C	R	R	C
real_estate.settlement_unit	C	R	R	C
real_estate.cost_share	C	C	C	C
real_estate.settlement_result	C	C	C	C
real_estate.bved.service_provider	C	—	R	C
real_estate.bved.provider_assignment	C	—	R	C
real_estate.bved.object_number	C	—	R	C
real_estate.bved.object_number. m_satz_line	C	—	R	C
real_estate.bved.export	C	—	R	C
real_estate.bved.import	C	—	R	C
real_estate.bved.import.line	C	—	R	C
account.configuration.real_estate	C	R	R	R

DATA MODEL

11.1 Property Management

real_estate.address (address.py)

Structured address with granular fields (street_name, building_number, unit_number, floor_number, room_number, post_box) plus an unstructured fallback text field. Supports both formats and is linked to base_object records.

real_estate.base_object (base_object.py)

Central entity for every real estate asset.

Types: property · building · object · land · equipment

Workflow: Draft □ Active □ Closed

Key features:

- Parent-child tree (tree() mixin), e.g. Property □ Building □ Apartment
- One2Many relations to Address, ObjectParty, Measurement, and BillingUnit
- History tracking via BaseObjectOccupancy (tenant occupancy periods)
- Meter readings (MeterReading) linked to equipment objects
- billing_as / collective_billing flags to control how operating cost billing is aggregated at property level
- next_billing_start_date (function field, property only): the earliest start_date among all non-billed billing units of the property. Drives the *ready for billing* checks and the green line-color highlight in the billing unit list view.
- Buttons compute_value_shares, compute_settlement_result_property, and ready_for_billing_property delegate bulk settlement actions to all billing units of the property that share next_billing_start_date. billing_property opens the real_estate.billing_unit.wizard instead of billing directly.
- call_billing / do_billing (classmethods) run the actual billing for one or more properties, optionally in the background queue (execute_in_queue). do_billing requires every billing unit that would be billed to already be in state ready_for_billing; with collective_billing all billing units sharing the same start date must be included together, otherwise a ValidationError is raised.
- bved_provider_assignments (function-less One2Many, readonly=True, visible only for type in ('property', 'building')) shows the real_estate.bved.provider_assignment records anchored on this object. The *BVED Provider Assignments* notebook page itself stays hidden until at least one such assignment actually exists — it is a read-only overview, not a way to create or edit assignments; those are always

maintained on the dedicated *BVED Provider-Liegenschaft* screen (`real_estate.bved.provider_assignment`, see below).

Rental object fields (visible only for type = 'object'):

type_of_use

Economic use: residential, commercial, property (owner-occupied), or internal. Drives the contract-type filter.

use_class

Many2One to `real_estate.use_class`. Controls which additional fields (`basement_nr` / `parking_nr`) appear on the form.

Meter fields (visible only for type = 'equipment', e_type = 'meters'):

meter_no_decimals

Boolean, default True. When set, meter readings are validated and rounded to integer values. The estimate-consumption wizard and `simulate_estimate` respect this flag and round accordingly.

Context classes for list views (`BaseObjectEquipmentContext`, `BaseObjectOccupancyContext`, `MeterReadingContext`) provide filterable search panels. Selection fields shared with the underlying model (e.g. `e_type`, `state`) are derived dynamically so their labels stay in sync automatically.

real_estate.object_party.role (object_party.py)

Configurable role definitions per object type (e.g. *Tenant* only for type `object`, *Owner* for all types).

real_estate.object_party (object_party.py)

Links a `party.party` record to a `BaseObject` with a typed role and an optional validity period.

real_estate.measurement.type (measurement.py)

Named measurement type linked to a `product.uom` unit.

Supports a **group hierarchy**: a type with `is_group` = True acts as a container grouping one or more child types via the parent Many2One field. Multi-level hierarchies are supported — groups can themselves have a parent group (e.g. *Gross Floor Area* □ *Net Floor Area* □ *Living Area*).

Constraints enforced at save time:

- All child types must share the same unit as their parent group.
- Circular references (a group referencing one of its own descendants as parent) are rejected.
- Only **leaf** (non-group) types can be assigned to individual `real_estate.measurement` records on `BaseObject` instances.

When a group type is referenced in `ContractTermType.m_type` or in `SettlementUnit.m_type`, the system automatically expands the group to all descendant leaf type IDs at query time, so measurements of any child type are included in the calculation.

get_effective_ids(m_type)

Classmethod resolving `m_type` to the flat list of leaf type ids to search: itself if not a group, else all descendant leaf ids (recursively, for nested groups).

real_estate.measurement (measurement.py)

Associates a numeric value and measurement type with a `BaseObject` for a given validity period.

get_total_value(base_object_id, m_type, as_of_date=None)

The single, hierarchy-aware entry point for “the value of `m_type` on `base_object_id` as of `as_of_date`” — every computation in the module that needs a measurement value (as opposed to just listing raw rows for display) goes through this classmethod rather than

querying `real_estate.measurement` directly. Resolves `m_type` via `MeasurementType.get_effective_ids()` and, for **each** effective leaf id independently, looks up the object's own latest row with `valid_from <= as_of_date` (or the single latest row ever, if `as_of_date` is `None`) — then **sums** whatever was found. This matters for a “Summenbemessung” (group type) where a single object carries values under **several sibling leaf types** at once (e.g. a mixed-use object with both a residential and a commercial area entry): the object contributes all of them, not just whichever happens to be the most recently dated row. Returns `None` if `m_type` is falsy, resolves to no effective ids, or none of them has any matching row at all (true “not recorded”, as opposed to a recorded value of zero) — callers use this to distinguish “no data” from a real zero. Used by `real_estate.bved.provider_assignment` (L-Satz *Gesamtfläche*), `BillingUnit.on_change_with_co2_total_area`, `SettlementUnit.compute_value_shares` (allocation by measurement), `ContractTerm._sum_measurements`, and `OptionRate._measurement_value` (dynamic option-rate weighting) — previously each of these re-implemented its own (in several cases incompletely hierarchy-aware, or not hierarchy-aware at all) lookup.

company / property (Function fields)

Display/search-only, derived from `base_object` — `company` is simply `base_object.company`; `property` is `base_object` itself if its own type = 'property', else `base_object.property` (the stored ancestor-property reference every `base_object` already carries). Used by the *Measurements* list below to show and filter by company/Wirtschaftseinheit without a join in the view itself. `search_property` matches either a row whose own property field equals the given value, or a property-type row whose own id does (so filtering by a given property also finds that property's own direct measurements, not just its descendants').

Measurements list (act_measurement_tree in measurement.xml)

Menu *Master Data*, between *Rental Object* and *Equipment*. A plain, filterable browse list over `real_estate.measurement` (columns: Company, Property, Object, From, Measurement Type, Value — summed —, Symbol), **not** a wizard and **not** hierarchy-aware the way `get_total_value()` is — it lists raw rows matching a simple filter, with no de-duplication to “the one currently effective row per (object, type)”. The filter panel (`real_estate.measurement.context`, via the action's `context_model/context_domain` — same mechanism as `real_estate.meter_reading.context` above) offers:

- `company` (required) — restricts to `base_object.company`.
- `property` — restricts the `base_object` picklist to that property's own subtree (via `child_of`) and, if `base_object` itself is left empty, restricts the list the same way.
- `base_object` — if set, takes precedence over `property` and restricts the list to this object and **all** of its descendants (('base_object', 'child_of', [base_object], 'parent') — inclusive of the object itself).
- `m_type` — exact match, if set.
- `date` (“Stichtag”, required, default today) — a plain `valid_from <= date` filter; deliberately **not** the latest-row-per-group logic of `get_total_value()` — every row satisfying the date cutoff is shown (and contributes to the summed Value column), including any historical rows later superseded by a newer `valid_from` on the same object/type.

real_estate.meter_reading (base_object.py)

Meter reading record linked to an equipment object of type meters.

Key fields: `company` (stored, auto-filled from `base_object` on change), `base_object`, `meter_id`, `reading_date`, `m_type` (initial / reading / estimate / final), `value`, `unit` (derived from the meter's `meter_unit`), `consumption` (difference to previous reading for counter meters), `comment`

(free text).

Browseable via the *Meter Readings* menu entry under *Master Data*, filterable by company, property, parent object, equipment (meters only), and date range (from_date / to_date).

Consumption estimate (simulate_estimate / create_estimate):

simulate_estimate(base_object, per_date, meter_id=None)

Returns (estimated_value, consumption, r1, r2). Prefers **interpolation** when readings exist both before and after per_date: takes the closest reading on each side and interpolates linearly between them. Falls back to **extrapolation** using the last two readings within one year before per_date when no future reading is available. Rounds the result to the meter's UOM digit precision; if meter_no_decimals is set on the meter, rounds to integers.

create_estimate(base_object, per_date, reason, meter_id=None)

Calls simulate_estimate and saves a new reading with m_type = 'estimate'.

11.2 Contract Management

real_estate.contract.type (contract_type.py)

Template for contracts. Defines invoice direction (in/out), default taxes, accounting journal, number prefix, step sizes for item and term sequence numbers, and whether occupancy exclusivity is enforced.

oc_mark

Free-text label used in operating cost settlement invoice descriptions and invoice headers, e.g. "Betriebskostenabrechnung 2025". Falls back to "Operating Cost Settlement" / "Operating Costs" when empty.

real_estate.contract.type.tax (contract_type.py)

Many2Many relation table between ContractType and account.tax.

real_estate.contract.term.type (contract_type.py)

Template for contract terms. Defines default rhythm, rhythm type (daily / weekly / monthly / quarterly / annually / one-time), rhythm start day, measurement type for quantity derivation, default quantity, and default account.

real_estate.contract (contract_core.py)

Main contract record.

Workflow: Draft ☐ Running ☐ Terminated / Cancelled

Key features:

- Links to party.party (contractual partner), base_object (property), and one or more ContractItem records
- Generates Tryton accounting moves (invoices) for all active terms via CreateContractMoves wizard (call_create_moves)
- Cash flow tabs on the contract form: *Draft* (invoice state draft/validated), *Pending* (posted), *Paid* (paid)
- _refresh_occupancy_for_contracts updates occupancy records when a contract starts or is terminated; cancelled contracts are excluded from occupancy calculations automatically
- add_log appends timestamped ContractLog entries
- next_item_sequence / next_term_sequence auto-increment helpers

settlement_units (Function field, get_settlement_units)

The contract's *last valid* settlement units: settlement units of the property's billing units whose objects overlap with the objects assigned to this contract via its items. Billing units are restricted to the property's `next_billing_start_date` (the earliest non-billed billing unit start date); if the property has none set, the most recently billed period is used instead. Used by `get_cost_shares` (cost shares of these settlement units belonging to the contract) and by `real_estate.contract.annex4.report` for the Anlage 4 print (see *Reports* below) — the report intentionally reuses this field instead of re-deriving the billing unit itself.

Fixed term / termination. `unlimited` (Boolean, default `True`) determines whether the contract has a fixed end date. Editable only in draft:

- `unlimited = True` (default) — `end_date` is readonly and must be empty; `on_change_unlimited` clears it automatically when toggled on.
- `unlimited = False` — `end_date` becomes required and editable (only in draft).
- The invariant (“unlimited contract must not have an end date”, `msg_contract_unlimited_with_end_date`) is enforced by `validate_fields` only in draft — once running/terminated it is no longer checked, since termination legitimately sets `end_date` regardless of `unlimited` (see below).

The *Terminate* button (`real_estate.terminate_contract.wizard`, see *Wizards*) now also writes the resolved `termination_date` into `end_date` — so `end_date` always reflects the contract's actual end once terminated, fixed-term or not.

Reactivating a **terminated** contract no longer uses the *Running* button (that button is now only shown for draft/cancelled). Instead, a dedicated *Revert Termination* button (`revert_termination`, same underlying `terminated → running` transition) is shown only for `state = 'terminated'`. Besides clearing the termination fields (`termination_date`, `terminated_by_type`, `receipt_of_termination_notice`, `termination_notice`, `termination_reason` — same as *Running* already did), it additionally clears `end_date` back to empty, but **only if the contract is “unlimited”** — a fixed-term contract that was terminated early keeps whatever is in `end_date` after being reactivated.

See also the `update_contract_status` cron task (*Configuration* above) for the automatic expired termination of fixed-term contracts whose `end_date` has passed without an active termination.

Cancellation: the *Cancel* button transitions the contract to *Cancelled*. Before the transition:

- If any cash flow entry in state `done` (already invoiced) exists, a `ContractCancelWarning` is shown explaining that existing postings will **not** be reversed automatically. The user must confirm to proceed.
- All cash flow entries still in state `draft` are deleted.
- Occupancy records are recalculated; the cancelled contract is excluded.

`real_estate.contract.log (contract_core.py)`

Append-only audit log attached to a contract (event name + description).

`real_estate.contract.item (contract_item.py)`

Associates a `base_object` of type `object` (e.g. an apartment) with a contract for a given validity period. On create/write/delete, triggers occupancy refresh and re-runs `BillingUnit` selection and value-share calculation for the affected property. Validates that `valid_from` lies within the contract period and raises a warning (or error for active contracts) on overlapping occupancy.

Each `ContractItem` can reference one or more rental objects via the `real_estate.contract.item.object` child model. The object selector always restricts to objects belonging to the **same property** as the contract (preventing cross-property assignments); it is additionally restricted to type

= 'object' only for **occupancy contracts** (contract.c_type.occupancy set — the Function field occupancy on ContractItemObject mirrors this from the contract type). Non-occupancy contract types may reference any base_object type (building, land, equipment, ...) belonging to the property. A unique SQL constraint on (item, object) prevents assigning the same object twice to the same item; the same object may still be assigned to a different item (e.g. on another contract).

real_estate.contract.term(contract_term.py)

A recurring charge line on a contract (rent, operating cost advance, etc.).

Key fields: term_type, reference_item, valid_from/valid_to, rhythm + rhythm_type + rhythm_start, quantity, unit, unit_price, taxes.

Key methods:

re_calc()

Rebuilds the CashFlow list from existing invoice lines and projects future entries up to _re_calc_year years ahead.

_next_document_date()

Calculates the next invoice date based on rhythm and last posting date.

_on_change_with_next_due_date()

Applies the contract's payment term to derive the due date.

real_estate.contract.term.tax(contract_term.py)

Many2Many relation table between ContractTerm and account.tax.

real_estate.contract.term.cash_flow(contract_term.py)

Projected or realised cash flow entry for a term.

States: draft (planned, no invoice line yet) · done (invoiced)

Invoice states (computed from the linked invoice): draft / validated — not yet posted (excluded from settlement calculations); posted — open receivable; paid — settled. Only posted and paid entries appear in the balance sheet and are used as *advance payment* in operating cost settlement.

Stores posting_date, document_date, due_date, and a link to the account.invoice.line once billed. base_object (function field) is derived from the linked invoice line and used to key advance payments to a (contract, object) pair during operating cost settlement. create_moves_run_id (format YYYYMMDD-HHMMSS-U<userid>) is set for all cash flow entries created in a single CreateContractMoves run, allowing traceability back to the wizard invocation. Supports date-pattern search (YYYY/MM or YYYY/MM/DD) in the name field.

Quantitative(contract_term.py)

Custom fields.Numeric subclass that carries a unit reference and sets quantitative=True in its field definition so the UI renders the associated unit symbol.

real_estate.contract.term.adjustment(contract_term.py)

History record documenting a single old-term → new-term adjustment (e.g. a rent/advance-payment change following an operating cost billing run, an operating cost plan, or a free percentage/absolute change).

States: draft · approved

Key fields: adjustment_mode (percentage / absolute), term_old (required, ondelete='RESTRICT'), term_new (ondelete='RESTRICT', set once the replacement term exists), and an optional settlement_result link when the adjustment originates from an operating cost billing run.

Function fields mirror company / property / contract from term_old, and valid_from / valid_to / amount / tax_amount / total_amount from both term_old (suffix _old) and

term_new (suffix _new), so a reviewer can compare the before/after amounts without opening either term individually.

Browseable read-only via the *Contract Term Adjustments* menu entry under *Contracts* ▢ *Adjustment*.

Note

Only the data model and its history/comparison fields exist so far — nothing currently creates these records. See the *Adjustment of Contract Terms* wizard (below, under *Wizards*), which defines the full input mask but still has placeholder processing logic only.

11.3 Operating Cost Settlement

real_estate.cost_category_group (billing_unit.py)

Groups cost types for reporting, e.g. *Heating, Water, Janitorial*.

real_estate.cost_type (billing_unit.py)

Individual cost item (e.g. *Gas, Cold Water*) with optional *category_group*, *comment*, and *no_print* flag.

For *allocation_by_consumption* settlement units, two fields control the meter-reading tolerance window around the start/end date of each cost share:

reading_pre_days

Days *before* the target date within which a reading is accepted (default: 7).

reading_post_days

Days *after* the target date within which a reading is accepted (default: 7).

The reading closest to the target date within the window is used. If no reading is found, an error is stored on the cost share.

real_estate.billing_unit (billing_unit.py)

Annual (or period) operating cost settlement for one property.

Workflow: Draft ▢ Approved ▢ Selection ▢ Value Share ▢ Ready for Billing ▢ Billed

external_billing

Boolean flag. When enabled, every settlement unit of the billing unit is forced to *allocation_rule = allocation_from_external_billing* (see *real_estate.settlement_unit* below) and costs are entered manually instead of being computed internally. Also reveals the *BVED* page and its *bved_provider_assignment* field — see *BVED External Billing Interface* below.

Two calculation methods:

rental_apartment

Operating cost settlement for residential tenancies under §§ 1–2 BetrKV. Costs are allocated to tenants proportionally.

WEG_billing

Annual statement for condominium owners under WEG (German condo law). Costs are allocated to co-owners by ownership share.

Key actions (buttons):

approved

Activates the billing unit.

selection

Identifies which contracts/objects are in scope for the billing period. Available in states approved, selection, value_share, and ready_for_billing (re-running from value_share / ready_for_billing revises the scope; existing settlement results are deleted after confirmation).

compute_value_shares_button

Calculates allocation shares (CostShare) for each settlement unit. Available in states selection, value_share, and ready_for_billing. If settlement results already exist for the billing unit, a confirmation warning is shown before they are deleted and value shares are recomputed. Does **not** automatically call compute_settlement_result. If any settlement unit has sub_state = error the billing unit is *not* advanced to value_share; the error is written to the billing unit log instead.

compute_settlement_result

Derives SettlementResult records per (contract, base_object) pair — actual costs, advances paid (only posted/paid invoice lines), and the resulting refund or additional receivable. Advance payments are matched to cost shares by the same (contract, object) key. If an advance-payment cash flow line has no matching cost-share group (e.g. no object on older invoices, or several terms for the same object) a fallback SettlementResult with actual_costs = 0 and a full refund is created for it; the fallback count is logged. Sets affected CostShare records to state error if any cash flow entries in the period are still in draft/validated state; the error message is prefixed with [draft] so it can be reset on re-run once the drafts are posted or deleted. Resets a billing unit already in state ready_for_billing back to value_share before recomputing.

check_ready_for_billing

Transitions value_share → ready_for_billing after validating:

- no settlement unit has sub_state = error;
- every settlement unit (except no_allocation) has reached sub_state = value_share;
- at least one approved SettlementResult exists and none has actual_costs = 0;
- every settlement result with a non-zero advance payment has both a term and a contract (a missing term usually means several term types contributed and the billing unit's term-type filter needs narrowing);
- all advance-payment invoices are posted;
- (unless external_billing) the sum of settlement result actual costs matches the sum of cost share actual costs.

Any failed check raises a ValidationError listing the offending records. Can also be triggered per property via BaseObject.ready_for_billing_property for all billing units sharing next_billing_start_date.

billing_wizard / billing

The form button billing_wizard opens the real_estate.billing_unit.wizard (see *Wizards* below), which calls the billing classmethod. billing creates, per settlement result, up to two invoice lines — advance-payment dissolution and actual costs — with taxes copied from the advance-payment term, then one invoice per contract. Invoices are created in state draft and posted immediately if the wizard's invoice_state is posted. Each invoice's payment_term (and therefore its due date) is taken from the wizard's payment_term field when set; otherwise the contract's own payment term is used, falling back to account.configuration's re_payment_term_billing. Both invoice_date and accounting_date are set to the wizard's invoice_date (analogous to

the `accounting_date` on contract invoices generated by `CreateContractMovesWizard`), so the posting date matches the invoice date rather than the date the wizard was run. Refuses to proceed if `sub_state` is error or if draft cash flow lines still exist in the settlement period; with `collective_billing` all billing units of the property sharing the same start date must already be `ready_for_billing`. Generates a `billing_run_id` (YYYYMMDD-HHMMSS-U<userid>) that is written to the billing unit and all `BillingUnitMoves` records of the same run, so every posting can be traced back to its run. Hidden when the property has `collective_billing = True`; in that case the action is triggered from the property form instead (`BaseObject.billing_property` `same wizard` `BaseObject.do_billing`).

billing_run_id

Char field set at the end of a successful `billing()` call, identical to the value written to all `BillingUnitMoves` of that run.

rental_objects

(Function, One2Many, readonly.) Distinct rental objects referenced by this billing unit's own `settlement_results` (via `settlement_result_objects()`) — the objects that actually ended up with a settlement result, as opposed to `bved_covered_object_ids()` (derived from `settlement_unit.objects` and used for BVED A-/D-Satz matching, which can include an object excluded during `selection()`, e.g. a genuine vacancy edge case). Shown as a read-only *Rental Objects* tab, right after *Settlement Results*.

covered_rental_objects()

Method (not a stored/Function field) used by the BVED L-Satz "Gesamtfläche" calculation — see *BVED External Billing Interface* below: returns `settlement_result_objects()` when non-empty, else falls back to the objects covered by this billing unit's own settlement units (`bved_covered_object_ids()`) for a billing unit that hasn't had "Compute Settlement Results" run yet. External callers (BVED included) always go through this one method rather than re-deriving the object set themselves.

real_estate.billing_unit.log (billing_unit.py)

Timestamped log entries attached to a billing unit.

real_estate.billing_unit.moves (billing_unit.py)

One record per invoice line created by `billing()` — a separate record for the advance-payment dissolution line and for the actual-costs line of the same settlement result (not combined into one record). Links the billing unit to the contract, settlement result, and the individual posting record:

- `moves_advanced_payment` — `account.invoice.line` that reverses the operating-cost advance (credit note line)
- `moves_actual_costs` — `account.invoice.line` for the actual costs
- `moves_alloc_by_owner` — `account.move.line` for the owner allocation journal entry

Function fields `amount`, `currency`, and `invoice` are derived from whichever of `moves_advanced_payment` / `moves_actual_costs` is set, for quick display in the tree view without opening the invoice line. For vacancy records (no contract, but `moves_alloc_by_owner` set), `amount` is instead computed as `debit - credit` of that `account.move.line`, `currency` from its own `currency` field, and `invoice` stays empty (move lines have no invoice reference).

`billing_run_id` ties all moves of one billing run together (same value as on the parent `BillingUnit`). Browseable via the *Billing Unit Moves* menu entry under *Operation Costs*, filtered by company, property, billing unit, contract, and date range.

real_estate.settlement_unit (settlement_unit.py)

One cost-type line within a billing unit, e.g. *Gas 2024*.

Allocation rules:

no_allocation

Entire cost stays unallocated.

allocation_by_measurement

Allocated by a measurement value, e.g. living area. $\text{value_share} = \text{measurement_value} \times \text{time_share} / \text{time_total}$ (time-weighted: a tenant occupying only part of the period receives a proportionally smaller share).

allocation_by_consumption

Allocated by meter reading consumption (HeizkostenV). $\text{value_share} = \text{raw consumption}$ (no time weighting applied).

Meter readings are looked up within a tolerance window defined by `reading_pre_days` / `reading_post_days` on the cost type; the closest reading to the target date is chosen.

Vacancy handling: cost shares without a contract receive $\text{value_share} = 0$ automatically — no meter reading is required. For the immediately following contract cost share, if no start reading exists within the normal window, the system falls back to the reading taken at the start of the preceding vacancy. This allows a single read-out at move-out to serve as both the vacancy baseline and the contract's opening reading.

Missing readings set the cost share to state error with one of:

- *“Messwert für Anfangsverbrauch nicht ermittelt”* — start reading absent
- *“Messwert für Endverbrauch nicht ermittelt”* — end reading absent

allocation_per_rental_unit

Proportional share by occupancy duration. $\text{value_share} = \text{time_share} / \text{time_total}$ (full period = 1.0, half period = 0.5, etc.).

allocation_from_external_billing

No internal calculation; `planned_costs/actual_costs` are entered manually on the cost share. Automatically suggested (via `on_change_external_billing`) on every settlement unit of a billing unit when that billing unit's `external_billing` flag is enabled (see `real_estate.billing_unit` below) — except units already set to `allocation_via_cost_collector`, which are left alone. Enforced (not just suggested) by `validate_fields`: for an externally billed billing unit, every settlement unit's `allocation_rule` must be either this or `allocation_via_cost_collector`.

allocation_via_cost_collector

No allocation of its own. Used to route one settlement unit's costs into another settlement unit of the *same* billing unit (its `reference_settlement_unit`, required for this rule and restricted to a unit that does not itself use this rule — no chaining) for allocation there instead. Also selectable when the billing unit has `external_billing` enabled, as an alternative to `allocation_from_external_billing` (e.g. several fuel suppliers, each invoiced on their own settlement unit, feeding one externally billed heating settlement unit).

A settlement unit with this rule:

- determines no rental objects of its own — objects always mirrors `reference_settlement_unit.objects`;
- generates no cost shares and is excluded from `Contract.get_settlement_units()` (so it never appears twice, once under its own cost type and once under the reference's, in e.g. the Annex 4 report);
- still tracks its own `planned_costs/actual_costs` (the latter from its own `invoice_lines`, via the normal `selection_actual_costs()`) — these are summed, across *all* settlement units referencing the same target, into the target's `planned_costs_from_references/actual_costs_from_references` (Function

fields, see below) and folded into the target's own `planned_costs/actual_costs` when it computes its value shares.

`BillingUnit.compute_value_shares_button` processes all `allocation_via_cost_collector` units of a billing unit before any other settlement unit, so a reference target always sees its referencing units' up-to-date actual costs. `BillingUnit.duplicate_next_period` copies `reference_settlement_unit` to point at the *newly created* successor of the original target (not the old period's unit) — non-cost-collector units are always copied first so that successor already exists by the time a referencing unit is copied.

planned_costs_from_references / actual_costs_from_references

(Function fields, any settlement unit) Sum of `planned_costs/ actual_costs` of all settlement units of the same billing unit that reference this one via `allocation_via_cost_collector`. Zero when nothing references this unit.

Vacancy handling: unoccupied periods can be charged to the owner or left unallocated.

When `compute_value_shares` is re-run, cost shares in state error are automatically reset to selection and their `error_message` cleared before the new calculation starts, so corrected readings or measurements take effect immediately.

Heating cost billing split (HeizkostenV §7/§8):

heating_billing_mode

Selection: none (default) / `central_heating` / `central_hot_water`. Independent of `allocation_rule/m_type` above (which govern how *this* settlement unit's own costs are allocated per tenant) — this models the legally mandated split, for a centrally supplied heating or hot water system, between the consumption-based and area-based portions of the heating cost statement itself.

heating_consumption_share_percent

Numeric percentage entered by the user: the share of heating/hot water cost allocated by consumption ("Anteil nach Verbrauchsumlage"). Required and visible only when `heating_billing_mode` is `central_heating` or `central_hot_water`; domain-restricted to 50–70 (statutory range, HeizkostenV §7/§8) — but only while one of those two modes is selected, so a value entered under a previous mode (or left over from an import) never blocks saving once the mode is set back to none.

heating_area_share_percent (Function field)

Computed as $100 - \text{heating_consumption_share_percent}$ ("Anteil nach Flächenumlage"); read-only, same visibility as above.

heating_area_measurement_type

Many2One to `real_estate.measurement.type` (object-level measurement types only): which measurement to use for the area-based portion of the heating cost split. Required and visible only under the same condition as the two fields above.

total_value_consumption / total_value_area

Sum, across all of this unit's cost shares, of their `consumption_share / area_share` (see `real_estate.cost_share` below). Read-only, visible only when `heating_billing_mode` is `central_heating/central_hot_water`. Computed by `_compute_heizkostenv_split()`, called at the end of `compute_value_shares()` (both the regular per-cost-share path and both branches of `_compute_value_shares_external()`) — i.e. whenever *Compute Value Shares* is run on the billing unit.

real_estate.cost_share (settlement_result.py)

Intermediate allocation record: share of a specific cost type for one contract or object within a billing period.

States: preparation · selection · estimated_value_share · value_share · error

Stores value_share (time-weighted allocation factor — for allocation_by_measurement and allocation_per_rental_unit this already incorporates the occupancy fraction time_share / time_total; for allocation_by_consumption it holds the raw consumption value, unless the settlement unit is HeizkostenV-split — see consumption_share below), time_share (days), planned_costs, actual_costs. error_message describes the problem; entries set by compute_settlement_result carry a [draft] prefix and are automatically cleared on the next successful re-run.

allocation_rule / external_billing (Function fields, mirrored)

Mirrored from the parent settlement_unit. planned_costs/actual_costs are only editable (readonly otherwise) when external_billing is set, i.e. for cost shares of a settlement unit with allocation_rule = allocation_from_external_billing.

heating_billing_mode (Function field, mirrored)

Mirrored from the parent settlement_unit; drives the visibility of the two fields below (only visible for central_heating/ central_hot_water).

consumption_share / area_share

The two HeizkostenV split components, computed by SettlementUnit._compute_heizkostenv_split() (called at the end of compute_value_shares()) only when the parent settlement unit's heating_billing_mode is central_heating/ central_hot_water:

- consumption_share — only for allocation_rule = allocation_by_consumption: the raw consumption value that would otherwise sit in value_share is moved here instead, and value_share is reset to 0 (so a HeizkostenV-split consumption-based unit currently distributes no planned_costs/actual_costs via compute_value_shares()'s own _distribute() step — a deliberate, temporary state pending a follow-up that recombines consumption_share/area_share via heating_consumption_share_percent/heating_area_share_percent into the actual cost distribution).
- area_share — computed for every cost share of a HeizkostenV-split unit regardless of its allocation_rule: the value of the unit's own heating_area_measurement_type for the cost share's base_object, weighted by time_share / settlement_unit.time_total (same weighting as allocation_by_measurement above).

Both values are summed into the parent settlement unit's total_value_consumption/total_value_area.

real_estate.settlement_result (settlement_result.py)

Final settlement record per contract (or object) and billing unit.

States: approved · billed

Stores planned_costs, actual_costs, advanced_payment (sum of posted/paid operating-cost advance invoice lines during the period), and the derived refund_receivable (positive = refund to tenant, negative = additional receivable). Optionally links to the single ContractTerm that covered all advances when exactly one term was involved. The display name includes the record id in parentheses (e.g. "2026-01-01 – 2026-12-31 / Mieter 1 (42)") to disambiguate several results for the same contract/object; the id is also searchable via the name field.

11.4 CO2 Cost Allocation (CO2KostAufG)

Implements the German *CO2-Kostenaufteilungsgesetz* (CO2KostAufG), which splits the CO2 cost of heating fuel between tenant and landlord depending on the building's emission intensity (kg CO2/m²/year). Applies via an optional `co2_kostaufg` reference on one or more of a billing unit's settlement units (typically its *Heizkosten* settlement unit) — settlement units without a reference are unaffected, and a billing unit with none of its settlement units referencing one gets no CO2 Costs page at all.

Note

The reference and the raw consumption-row table live on `real_estate.settlement_unit` (unchanged from earlier versions), but every *derived* figure (totals, area, per-m² emission, tenant/landlord shares) is aggregated on the parent `real_estate.billing_unit` instead — across **all** of its settlement units that carry a `co2_kostaufg` reference, not just one. This matters when a billing unit's costs are split across more than one CO2KostAufG source (e.g. two heating circuits, two fuel deliveries under separate master records): the billing unit's total consumption/emission/cost figures, and the object set used for the area basis, are the union across all of them, not just whichever settlement unit happens to be entered first.

`real_estate.co2_kostaufg (co2_kostaufg.py)`

Master record for one CO2KostAufG data source (e.g. one heating installation/fuel), scoped to a property (required, type = 'property') with company derived from it (Function field). Referenced from one or more settlement units of the same property via their `co2_kostaufg` field (domain-restricted to that property — a settlement unit can only pick a CO2KostAufG belonging to its own property). Menu: *Real Estate* ▢ *Operation Costs* ▢ *CO2 Kosten verwalten*.

`real_estate.co2_kostaufg.consumption (co2_kostaufg.py)`

Child records under a `co2_kostaufg` (parent, `ondelete='CASCADE'`), one per billed/metered period: `date_from/date_to`, `consumption_kwh`, `co2_kg_per_kwh` (emission factor), `co2_emission_kg`, `co2_price_ct_per_kwh`, `vat_rate`, `co2_cost_net`, `co2_cost_gross`.

`co2_emission_kg` is pre-filled (`consumption_kwh × co2_kg_per_kwh`) only while still empty — once any value exists (auto-filled or typed by the user), later changes to `consumption_kwh/co2_kg_per_kwh` never overwrite it again, so it is always freely (re-)editable, e.g. to enter the supplier's own figure. `co2_cost_net/co2_cost_gross` behave differently **on purpose**: they are always recomputed from `consumption_kwh/co2_price_ct_per_kwh/vat_rate` whenever any of those three change; a manual override of net/gross only sticks until the next such recompute.

`split`

Boolean. Set automatically by `SettlementUnit.compute_value_shares()` when this record was created by splitting an original record at a settlement-period boundary — see *Splitting at settlement-period boundaries* below.

Uses `DeactivableMixin` — a record replaced by a split is deactivated (`active = False`), never physically deleted.

`real_estate.co2_emission_share (co2_kostaufg.py)`

Configuration table for the legal 10-tier distribution model (Anlage 1 CO2KostAufG, residential rented buildings without separate sub-metering by usage type): `emission_limit` (kg CO2/m²/year — the *exclusive* upper bound of the tier), `tenant_share / landlord_share` (% , must sum to 100 — enforced in `validate()`). `get_share(value)` returns the first row, in sequence order, whose `emission_limit` is strictly greater than `value`; if `value` reaches or exceeds every configured limit, the last row (by sequence) is used as an open-ended top tier regardless of its own stored limit value. Default data for all 10 legal tiers is loaded at module installation (< 12 kg ▢ 0 % / 100 %

landlord/tenant ... $\geq 52 \text{ kg} \cdot 95 \% / 5 \%$). Menu: *Real Estate* $\square$ *Configuration* $\square$ *Emission-CO2 Verteilung*.

Fields kept on `real_estate.settlement_unit` — a minimal CO2 Costs page shows just these two:

co2_kostaufg

Many2One reference, restricted to CO2KostAufG records of the same property as the settlement unit.

co2_consumption

(Function, One2Many, readonly.) All consumption records of the referenced CO2KostAufG that overlap *this settlement unit's own period* — the raw, per-source audit table.

The boundary-splitting logic (`_split_co2_consumption()`, see *Splitting at settlement-period boundaries* below) still operates per settlement unit and its own `co2_kostaufg/period`, unaffected by the aggregation described next.

Fields added to `real_estate.billing_unit` — a CO2 Costs page is shown only when `co2_relevant` is true (i.e. at least one of the billing unit's settlement units has `co2_kostaufg` set); a *Non-residential building >50% commercial (§8)* flag at the top of the page then further splits which of the remaining fields are shown:

co2_relevant

(Function.) True if any settlement unit of this billing unit has `co2_kostaufg` set. Controls the page's own visibility.

non_residential_flag

Boolean, default **false**. CO2KostAufG §8: sets whether this building is treated as predominantly commercial (>50 % of usable area) — where the residential emission-per-m² tier table (Anlage 1) does not apply and a flat, configured tenant/landlord split is used instead. This is also the corresponding BVED L-Satz field 19 (Kennzeichen Nichtwohngebäude) — see *BVED External Billing Interface* below, whose provider-assignment L-Satz preview and real export both read this field directly (no longer derived from the Optionssatz).

- **False** (default): `co2_measurement_type`, `co2_consumptions`, the four totals, `co2_emission_per_m2`, and the residential-tier `co2_tenant_share/co2_landlord_share` are shown; the two commercial share fields are hidden.
- **True**: only `co2_commercial_tenant_share/co2_commercial_landlord_share` are shown; all of the above are hidden. (The underlying values are always computed regardless of the flag — it only controls what the form displays.)

co2_measurement_type

Many2One to `real_estate.measurement.type` (object-type measurements only) — which measurement to sum for the area basis below, across the union of every co2-relevant settlement unit's own objects (each object counted once even if covered by more than one such settlement unit).

co2_consumptions

(Function, One2Many, readonly.) Union of the consumption records of every `co2_kostaufg` referenced by any settlement unit of this billing unit, overlapping the billing unit's own `start_date/end_date` — the aggregated audit table (as opposed to each settlement unit's own single-source `co2_consumption` above).

co2_total_consumption / co2_total_emission / co2_total_cost_gross

(Function.) Sum of `consumption_kwh / co2_emission_kg / co2_cost_gross` across `co2_consumptions` above, each weighted by the fraction of the record's *own* date range that actually falls inside the billing unit's period. A record fully inside the period contributes 100 %; a record extending beyond either boundary is interpolated pro-rata by days, always

relative to its own actual range — a record that simply doesn't reach as far as the period's end (no later data booked yet) is never extrapolated, it only ever contributes its own days.

co2_total_area

(Function.) Sum, across the union of objects covered by every co2-relevant settlement unit (deduplicated), of each object's latest `real_estate.measurement` value of `co2_measurement_type` (or its effective leaf types, if a measurement group) valid as of the billing unit's `end_date`.

co2_emission_per_m2

(Function.) `co2_total_emission / co2_total_area`, annualized via $\times 365 / (\text{end_date} - \text{start_date} + 1)$ so that billing periods shorter or longer than a calendar year still yield a correct "kg CO₂/m²/year" figure.

co2_tenant_share / co2_landlord_share

(Function.) Looked up via `Co2EmissionShare.get_share(co2_emission_per_m2)` — the residential 10-tier split. Always computed; only shown when `non_residential_flag` is false.

co2_commercial_tenant_share / co2_commercial_landlord_share

(Function.) The flat split used for commercial properties (not covered by the residential tier model): the company's configured `re_accounting.co2_landlord_share_commercial` (landlord), and 100 % minus that value (tenant). Always computed; only shown when `non_residential_flag` is true.

Splitting at settlement-period boundaries. `SettlementUnit.compute_value_shares()` ("Compute Value Shares" button, also reachable via the billing unit's own button of the same name) calls `_split_co2_consumption()` first, before anything else. For each of `start_date` and `end_date` + 1 day, every active consumption record of the referenced CO₂KostAufG whose own `[date_from, date_to]` strictly contains that boundary date is split into two new records at that date: amounts are interpolated pro-rata by day count (the second part is computed as the remainder of the first, so the two new records always sum back exactly to the original — no rounding drift); rate fields (`co2_kg_per_kwh`, `co2_price_ct_per_kwh`, `vat_rate`) are copied unchanged onto both. Both new records get `split = True`; the original is deactivated (soft-deleted via `active`), never physically removed. A record entirely inside the settlement period, or one that only covers part of it because no later data has been booked yet, is never split or extrapolated — only an *existing* record that actually spans across a period boundary gets divided. Re-running is idempotent: already boundary-aligned active records are left untouched, so repeated calls never produce duplicate splits.

11.5 BVED External Billing Interface

Implements the BVED / ARGE-FHW "Standard-Datenaustausch" Version 3.10 — the German fixed-width record format used to exchange operating-cost data with an external Messdienstleister (heating-cost/consumption billing service) — for billing units with `external_billing = True` (see *Operating Cost Settlement* above). Exports the property/tenant master data plus optional fuel and cost data the provider needs to bill (A-/L-/M-/B-/K-Satz); imports the provider's response (D-Satz actual costs, plus optional E835-/E898-/P-Satz) and applies it back onto `real_estate.settlement_result`. Implemented in `bved.py` (Tryton models) and `bved_records.py` (pure-Python fixed-width (de)serialization with no Tryton dependency, unit-testable standalone).

Menu: *Real Estate* ▢ *Operation Costs* ▢ *Externe Abrechnung* (placed immediately before *Betriebskostenabrechnung*), bundling all models below.

real_estate.bved.unit (bved.py)

Reference table mirroring BVED Tabelle 'E' ("Einheiten/Maßeinheiten", see `bved_records.TABLE_E`) — code (3-digit key, e.g. '010'), description (e.g. 'm² Wohnfläche'), and

name (Function, "{code} – {description}", e.g. '010 – m² Wohnfläche'). All 29 entries from the table are loaded as default data at module installation, with measurement_type left unset — mapping a unit to a real_estate.measurement.type (e.g. '010 – m² Wohnfläche' → the “Living Space” measurement type) is a per-database configuration decision, made via the *BVED Units* list (menu: *Real Estate* → *Operation Costs* → *Externe Abrechnung* → *BVED Units*). Used by real_estate.bved.provider_assignment.allocation1_unit/allocation2_unit/allocation3_unit (below) to resolve M-Satz fields 36-41.

real_estate.bved.service_provider (bved.py)

The Messdienstleister itself: party, bved_key (2-char code from BVED Tabelle ‘U’, e.g. '40' for ista — not enforced as a fixed selection since the table changes over time), bved_version (currently only '3.10'), transport_type (only 'manual' file download/upload is actually implemented; email/sftp/webbservice_api are reserved for a future automated transport layer).

M-Satz field 7 (Kennzeichen Adressfeld) and address-source rules:

field7_mode

Selection '1'=Nutzer / '2'=Eigentümer / '3'=Eigentümer/Nutzer / '4'=Leistungsnehmer (default '1', matching the field's old hardcoded value). Written unchanged into M-Satz field 7 of every record generated for this provider, and controls which address block(s) BvedObjectNumber._m_satz_values() actually fills:

- '1'/'3' — fields 8-15 (Nutzer) from the occupying contract's own party (entry. contract.contractual_partner).
- '2'/'3' — fields 16-23 (Eigentümer) per owner_rule.
- '4' — fields 59-66 (Leistungsnehmer) per tenant_rule.

owner_rule

Selection 'company'=Gesellschaft ist Eigentümer / 'role'=Eigentümer über Rolle (default 'role'). Visible/ required only when field7_mode is '2'/'3', but always *evaluated* (see provider_org_rule below) regardless of field7_mode. 'company': the Eigentümer block is filled from the company party/address (the same ones passed as company_party/company_address). 'role': filled via owner_role instead.

owner_role

Many2One to real_estate.object_party.role — which role counts as “Eigentümer” (used when owner_rule = 'role'), resolved via BvedObjectNumber._find_party_by_role() below. Defaults to the module's built-in “Owner” role (object_party_owner_role) for new records; a provider that needs a different role (e.g. one that also covers building/land, since the built-in role only covers property/object) can override it here.

tenant_rule

Selection 'tenant'=Mieter / 'role'=über Partner-Rolle (default 'tenant'). Visible/required only when field7_mode is '4'. 'tenant': the Leistungsnehmer block (59-66) is filled the same way as the Nutzer block (the occupying contract's own party — reproducing the module's original, unconditional “debtor mirrors tenant” behavior). 'role': filled via tenant_role instead.

tenant_role

Many2One to real_estate.object_party.role — which role counts as “Leistungsnehmer” (used when tenant_rule = 'role'), looked up the same way as owner_role.

provider_org_rule

Selection 'owner'=Leistungsgeber wie Eigentümer / 'role'=über Partner-Rolle (required, default 'owner'). Fills M-Satz fields 42-49 (Leistungsgeber name/address) — **independent of**

field7_mode (fields 42-49 are always attempted, regardless of what field 7 selects). 'owner': reuses whichever party/address owner_rule resolved to, even if field7_mode itself isn't '2'/'3' (so a provider can have field 7 = "Nutzer" while still deriving the Leistungsgeber block from the configured Eigentümer source). 'role': filled via provider_org_role instead.

provider_org_role

Many2One to real_estate.object_party.role — which role counts as "Leistungsgeber" (used when provider_org_rule = 'role'), looked up the same way as owner_rule.

tax_id_type

Selection sourced from party.identifier.get_types() (all globally-configured identification types, e.g. 'de_vat') — which type on the company party (Leistungsgeber, the same party used for M-Satz fields 42-49) provides M-Satz field 51 (USt-ID-Nr. oder Steuernummer). When set and the company party has a matching party.identifier, field 51 gets that identifier's code and field 50 gets tax_id_flag below; both fields stay unset if left empty or no matching identifier exists.

tax_id_flag

Selection '1'=USt-ID-Nr. / '2'=Steuernummer (visible/ required only when tax_id_type is set; default '2', matching the field's old hardcoded value). Written into M-Satz field 50 (Kennzeichen USt-ID/Steuernummer) whenever field 51 is — the admin's own classification of what the configured tax_id_type actually represents, since Tabelle 'E' has no generic notion of "this is a USt-ID vs. a Steuernummer" built in.

tax_rate_flag

Selection '1'=Regelsteuersatz / '2'=ermäßigt, optional (Kann-Feld). Written unchanged into M-Satz field 52 (Kennzeichen Steuersatz) of every record for this provider; left unset, field 52 stays unset too.

invoice_number_flag

Selection '0'=keine Rechnung §14 UStG / '1'=aus Feld 54 / '2'=vom Abrechnungsunternehmen erstellt, optional (Kann-Feld). Written unchanged into M-Satz field 53 (Kennzeichen Rechnungsnummer). '1' additionally requires invoice_number_rule to fill field 54 itself.

invoice_number_rule

Selection 'contract'=Nr. Mietvertrag / 'object'=Nr. Mietobjekt / 'settlement_result'=ID Settlement result. Visible/required only when invoice_number_flag is '1'. Fills M-Satz field 54 (Rechnungsnummer, max. 25 chars) per occupancy segment: 'contract' — the occupying contract's own contract_number; 'object' — this mapping's own base_object.object_number; 'settlement_result' — the id of the real_estate.settlement_result matching this object (and contract, or None for a vacancy segment) whose billing_unit period overlaps [period_start, period_end]. Field 54 stays unset if no matching value/record is found.

direct_debit_flag

Selection '0'=keine Abbuchungserlaubnis / '1'=Abbuchungserlaubnis, default '0'. Written unchanged into M-Satz field 58 (Kennzeichen Zahlungsart) of every record for this provider.

Note

owner_rule, tenant_rule, invoice_number_rule, tax_id_flag, tax_rate_flag, invoice_number_flag, and direct_debit_flag all include an explicit blank ('', '') choice, even where a default value is set — a plain optional fields.Selection with a static choice list otherwise rejects None/'' as "not a valid selection" on save once the field's own default no longer applies (e.g. after the value is cleared again in the client). Fields that are unconditionally required=True (field7_mode, provider_org_role) don't need this, since a blank value is

never legal for them anyway.

real_estate.bved.provider_assignment (bved.py, form label “BVED Provider-Liegenschaft”)

Anchors one provider + customer number + 9-digit Liegenschaftsnummer (external_property_number) to a base_object of type property or building, with valid_from/valid_to so a provider change over time doesn’t destroy history. Unique per (base_object, provider, valid_from) — a property/building may have several *concurrent* assignments for different providers (e.g. heating via Techem, water via ista, both on the same building). Referenced explicitly from billing_unit.bved_provider_assignment (never derived from the object tree), since one building can have more than one active assignment and only the user knows which one a given billing unit means.

object_numbers

One2Many to real_estate.bved.object_number (below).

allocation1_unit / allocation2_unit / allocation3_unit

(“M-Satz Allocation Key 1/2/3”) Many2One to real_estate.bved.unit each. Resolve M-Satz fields 36/37, 38/39, 40/41 (Schlüssel/Anteil Umlage 1-3) for every object number mapping under this assignment, via BvedObjectNumber._allocation_shares(): field 36/38/40 gets the unit’s own code (Tabelle ‘E’), field 37/39/41 gets the rental object’s own measurement value (as of the M-Satz row’s occupancy_end) for the unit’s mapped measurement_type — left unset if the unit has no measurement_type mapped (the key is still written on its own in that case) or if the slot itself is left empty.

tenant_change_fee_flag

Selection '0'=keine Umlage / '1'=Umlage (required, default '0'). Written unchanged into M-Satz field 68 (Kennzeichen Umlage Nutzerwechselgebühr) of every record generated for this provider assignment — unlike the other M-Satz catalog settings introduced so far (all on real_estate.bved.service_provider), this one lives on the assignment itself, since whether a tenant-change fee is allocated can plausibly differ per property/Liegenschaft even under the same provider.

L-Satz Preview page — read-only preview of how this assignment’s L-Satz (Liegenschaft) record would look today, split into two field groups:

- **Automatic** — l_period_start/l_period_end (always the most recently completed calendar year), l_provider_key, l_street/ l_country/l_postal_code/l_city (from the property’s/ building’s address), l_vat_flag (from the covered billing units’ Optionssatz: 100 % ☐ net/fully opted, 0 % ☐ no VAT shown, mixed ☐ per M-Satz field 25/user), l_weg_flag (from calculation_method — Cash basis sets it, Accrual basis doesn’t), l_non_residential_flag (true if any covered billing unit’s own non_residential_flag is set — see *CO2 Cost Allocation* above), l_co2_landlord_share_percent (from the covered heating-cost billing unit’s own co2_landlord_share/co2_commercial_landlord_share), l_total_area (computed once a measurement type is selected below).
- **To Be Entered** — gross_floor_area_measurement_type (drives l_total_area: summed, as of each billing unit’s own end date, over the union of every assigned billing unit’s own covered_rental_objects() — objects with a settlement result where one exists, else every object covered by a settlement unit; each object counted once even if covered by several billing units), plus six purely-manual fields with no derivable source: vacancy_risk_flag/vacancy_risk_percent, labor_share_flag, energy_improvement_flag, heat_supply_flag, heat_connection_2023_flag (various L-Satz Kann-/Muss-Felder — legal facts about the building not represented anywhere else in the data model). vacancy_risk_flag is also reused, unchanged, for M-Satz field 26 (Kennzeichen Umlageausfallwagnis) — see BvedObjectNumber._m_satz_values() below.

The preview always uses the most recently completed calendar year across every billing unit currently assigned; the real export (`BvedExport._build_l_m_records()`) recomputes the same underlying helpers per specific billing unit and its own period instead, so preview and actual export can differ if a billing unit's period doesn't align with a plain calendar year.

`real_estate.bved.object_number (bved.py)`

Maps one rental object to its `internal_reference` ("Ordnungsbegriff des Auftraggebers") and 4-digit `external_unit_number` under a provider assignment, with its own `valid_from/valid_to` — a provider renumbering (e.g. after a renovation) should close the old row and add a new one rather than overwrite `internal_reference` in place, otherwise already-imported historical D-Satz lines referencing the old number can no longer be resolved. Numbers are assigned automatically (`BvedExport._ensure_object_numbers()`) the first time an object is exported — scoped per assignment (not per billing unit) and counted all-time, so a closed mapping's number is never reused by a later one.

`_m_satz_values(period_start, period_end, provider, company_party=None, company_address=None, warnings=None, refresh_occupancy=True)`

Returns the list of M-Satz value dicts for this mapping's object over the given period — one dict per `real_estate.base_object.occupancy` segment overlapping it (vacancy periods included with `vacancy_flag=1`). Field 25 (Kennzeichen MwSt, `vat_treatment_flag`) is derived per segment from the object's own Optionssatz as of that segment's `occupancy_end`: 0 (kein Ausweis) if 0 % (or unknown), else 1 (gewerbl. Vermietung). Field 7 (`address_flag`) and the Nutzer/Eigentümer/Leistungsnehmer/Leistungsgeber blocks are governed by `provider.field7_mode/owner_rule/tenant_rule/provider_org_rule` — see `real_estate.bved.service_provider` above for the full rule description. Bank account fields (55/56) and `vacancy_flag` keep coming unconditionally from the occupying contract's own party, independent of `field7_mode`. Field 51 (USt-ID-Nr. oder Steuernummer) comes from `company_party`'s own `party.identifizier` of the type configured on `provider.tax_id_type` — always the company party specifically, never the `field7_mode/provider_org_rule`-resolved Leistungsgeber party; field 50 (Kennzeichen USt-ID/Steuernummer) is `provider.tax_id_flag` whenever field 51 is set, otherwise both fields stay unset. Field 52 (Kennzeichen Steuersatz) is `provider.tax_rate_flag` unchanged, if set. Fields 53/54 (Kennzeichen Rechnungsnummer / Rechnungsnummer) come from `provider.invoice_number_flag` and, when that is '1', `provider.invoice_number_rule`. Field 58 (Kennzeichen Zahlungsart) is `provider.direct_debit_flag` unchanged (see `real_estate.bved.service_provider` above for all of these). Field 68 (Kennzeichen Umlage Nutzerwechselgebühr) is `assignment.tenant_change_fee_flag` unchanged — this one comes from the provider *assignment* (`real_estate.bved.provider_assignment` above), not the provider itself. Field 26 (Kennzeichen Umlageausfallwagnis) is 1 if `assignment.vacancy_risk_flag` (the assignment's own L-Satz field 13) is set, else 0 — reusing the L-Satz value directly rather than a separate M-Satz-specific setting. The single, shared implementation of the M-Satz record content — used both by the real export (`BvedExport._build_l_m_records()`) and by `refresh_m_satz_preview()` below, so they can never diverge. `refresh_occupancy` (default True) recomputes `real_estate.base_object.occupancy` first.

`_find_party_by_role(ObjectParty, role_id, as_of_date)`

Generic role-based party lookup — shared by the Eigentümer (`owner_role`), Leistungsnehmer (`tenant_role`), and Leistungsgeber (`provider_org_role`) resolutions alike: searches `real_estate.object_party` for `role_id`, valid *exactly on* `as_of_date` (a point-in-time check — `valid_from` <= `as_of_date` and no `valid_to` or `valid_to` >= `as_of_date` — not merely overlapping the billing period), walking up from this mapping's own `base_object` through its parent chain (rental object → building/land → property) until a match is found or the top of the tree is reached. Returns None immediately if `role_id` is None. All three call sites in `_m_satz_values()` use `period_end` as `as_of_date` (resolved once per call, not per occupancy segment).

_area_share_by_mode(mode, as_of_date)

Shared by `_heating_base_share()` (mode='central_heating', M-Satz fields 27/69) and `_hotwater_base_share()` (mode='central_hot_water', fields 30/70). Returns (share, key):

- share — the object's own **direct** measurement value (via `Measurement.get_total_value()`, as of `as_of_date`) for the `heating_area_measurement_type` of the (first, if several) settlement unit belonging to this mapping's object's property with `heating_billing_mode = mode`. Deliberately **not** time-weighted by the occupancy segment (unlike `cost_share.area_share`, which drives the actual cost allocation and is prorated by `time_share/time_total`) and **not** affected by the `heating_consumption_share_percent/heating_area_share_percent` split — fields 27/30 always report the object's plain Bemessung. Does not require `compute_value_shares()` to have been run at all (no cost share involved). None — leaving the field unset (Kann-Feld) — if no qualifying settlement unit has a `heating_area_measurement_type` set, or the object has no matching measurement recorded.
- key — the BVED Tabelle 'E' code (see `real_estate.bved.unit` above) of the unit whose own `measurement_type` matches that measurement type — i.e. the unit share is actually expressed in. None if none is mapped.

`_heating_base_share(as_of_date)` and `_hotwater_base_share(as_of_date)` are thin wrappers writing into fields 27/69 and 30/70 respectively (called with the M-Satz row's own `occupancy_end` as `as_of_date`) — the two are fully independent (a property can have both a central-heating and a central-hot-water settlement unit, each with its own `heating_area_measurement_type`/mapped `real_estate.bved.unit`).

_mode_settlement_unit(mode)

The first `real_estate.settlement_unit` of this mapping's own object's property with `heating_billing_mode = mode`, or None — the same lookup `_area_share_by_mode()` performs inline, factored out since `_advance_by_mode()` below needs it independently of any `heating_area_measurement_type`.

_advance_by_mode(mode, contract, start_date, end_date)

M-Satz fields 28/29 (Heizung Vorauszahlung Brutto/Netto, mode='central_heating') or 31/32 (Warmwasser Vorauszahlung Brutto/Netto, mode='central_hot_water') — **not** 30/31, which is the Warmwasser Grundanteil/Vorauszahlung-Brutto pair respectively; field 30 itself is the (already-implemented) Warmwasser Grundanteil, see `_area_share_by_mode()` above. Returns (gross, net), either possibly None:

- Resolves the qualifying settlement unit via `_mode_settlement_unit(mode)` and, through it, its own `billing_unit` — the billing unit whose *Konditionstyp* (`term_types_of_use`) and *Vorauszahlungen* tab (`cash_flow_lines`) define which advance-payment condition applies. (None, None) if no qualifying settlement unit, or it has no `billing_unit`.
- For mode='central_hot_water': if the property's own central-heating settlement unit exists and shares the *same* `billing_unit` as the central-hot-water one, returns (None, None) unconditionally — German rental contracts normally carry a single combined "Heizkosten" advance payment covering both heating and hot water; attributing it to both fields would double it. Sharing a billing unit is the normal setup where one annual "Heizkosten" cost group contains both a central-heating and a central-hot-water settlement unit side by side (see `tests/test_billing_unit.py`'s two-Settlement-Unit "Heizkosten" billing unit).
- Otherwise, sums `total_amount` (gross) and `amount` (net) over every entry of the billing unit's own `cash_flow_lines` (already filtered there by `term_types_of_use/invoice`

state, see *Operating Cost Settlement* above) whose own contract matches contract and base_object matches this mapping's own object, restricted to document_date within [start_date, end_date]. (None, None) if nothing matches (a genuine Kann-Feld, not a zero).

_heating_advance(contract, start_date, end_date) and
_hotwater_advance(contract, start_date, end_date) are thin wrappers, called from _m_satz_values() with the M-Satz row's own occupancy_start/occupancy_end (not the whole export period) and its resolved tenant contract (None for a vacancy segment, in which case both return (None, None) immediately).

Note

Fields 33-35 (Kaltwasser Grundanteil/Vorauszahlung) already exist as Kann-Felder on real_estate.bved.object_number.m_satz_line (coldwater_base_share/coldwater_advance_gross/_net) but are not populated by _m_satz_values() — there is no heating_billing_mode value for a centrally-billed cold-water system (cold water is always metered per object via allocation_by_consumption in this module), so _mode_settlement_unit()/_advance_by_mode() have no analogous “central cold water” case to resolve them from.

_allocation_shares(as_of_date)

Resolves M-Satz fields 36-41 (Schlüssel/Anteil Umlage 1-3): for each of the provider assignment's allocation1_unit/ allocation2_unit/allocation3_unit slots (in order), returns (code, share) — code is the real_estate.bved.unit's own Tabelle 'E' code, share is this mapping's own base_object's measurement value (via Measurement.get_total_value(), no time-weighting) for the unit's measurement_type as of as_of_date — or (code, None) if the unit has no measurement_type mapped, or None for an unset slot. Always returns exactly 3 entries. Called with each M-Satz row's own occupancy_end as as_of_date.

m_satz_lines / refresh_m_satz_preview

A stored (not Function) One2Many to real_estate.bved.object_number.m_satz_line (below), populated by the refresh_m_satz_preview button: deletes any existing preview lines for the mapping, then rebuilds them from _m_satz_values() for the most recently completed calendar year (same period as the provider assignment's L-Satz preview, BvedProviderAssignment._last_full_year()) — one line per occupancy segment. Deliberately a real, explicitly-triggered button rather than a Function field: a Function getter runs inside a *read-only* database transaction (plain read() RPCs), where _m_satz_values(refresh_occupancy=True)'s delete-and-recreate inside Occupancy.refresh() would fail outright — and a true multi-row, per-field table needs real addressable child records for Tryton's list/form widgets anyway.

real_estate.bved.object_number.m_satz_line (bved.py)

One row per occupancy segment, generated only by BvedObjectNumber.refresh_m_satz_preview() — never created or edited directly (every field is readonly). Holds all 70 M-Satz fields from the BVED Nutzer/Eigentümer-Satz spec (field number prefixed in each field's label, e.g. “8. Tenant Name 1”), grouped on the form by spec section (identification, tenant, owner, occupancy period, VAT/vacancy-risk/heating-hotwater-coldwater shares, allocations, provider organization, tax/invoice/bank/payment, debtor, other flags/keys). The embedded list view shows just occupancy_start/occupancy_end/vacancy_flag/tenant_name1 for a quick overview across several tenants/vacancy periods; opening a row shows the full detail form.

Fields added to real_estate.billing_unit:

bved_provider_assignment

Which provider assignment applies (domain: same property/building subtree, validity window covering the billing unit's start date). Only visible when `external_billing` is set.

bved_object_numbers

(Function, One2Many.) The assignment's object numbers, restricted to objects actually covered by this billing unit (`bved_covered_object_ids()` — type object base objects reachable via at least one non-no_allocation settlement unit).

check_bved_provider_assignment()

Validation (run from `pre_validate`) rejecting a billing unit whose covered objects include one that is neither the assignment's own base_object nor a descendant of it (e.g. an object from a different building than the one referenced).

Shown on a dedicated *BVED* page, visible only when `external_billing` is set.

Fields added to `real_estate.settlement_unit` (B-Satz — fuel/stock/hot-water data, typically entered only on the heating-cost settlement unit):

bved_fuel_data

Enables the rest of the B-Satz fields below; also identifies “the” heating-cost unit representative of a billing unit's CO2 landlord share (see *CO2 Cost Allocation* above).

`bved_fuel_type` (BVED Tabelle ‘B’), `bved_heating_value`, `bved_fuel_indicator` (field 23, Kennzeichen Brennstoffart — Selection ‘0’-‘9’, default ‘0’), and `bved_fuel_rule` (‘stock’/‘meter’, default ‘stock’) — selects which of two mutually exclusive field groups below the *B-Satz / Brennstoff* page shows; two further groups apply either way and are always shown.

`bved_fuel_indicator` distinguishes several concurrent B-Satz fuel records for the same property/period (e.g. two fuel suppliers each feeding their own externally-billed settlement unit within the same billing period). ‘0’ if field 8 (`bved_fuel_type`) is left empty, else ‘1’-‘9’. `on_change_bved_fuel_type` keeps it consistent automatically: clearing field 8 resets it to ‘0’; setting field 8 bumps it from ‘0’/empty to ‘1’ (the user may raise it to ‘2’-‘9’ by hand for an additional concurrent fuel record). `SettlementUnit.validate_fields` enforces, whenever `bved_fuel_data` is set: the ‘0’ ⇔ empty-field-8 correspondence above, and — for a non-‘0’ value — uniqueness across every other settlement unit of the same property (`billing_unit.property`) whose billing period overlaps this one's (`billing_unit.start_date/end_date`).

‘stock’ (Bestandsführung, B-Satz fields 10-17 — storable fuels like heating oil, tracked via initial/closing stock only): stock start/end date + quantity + amount (gross/net). Hidden when `bved_fuel_rule = ‘meter’`.

WW-Anteil (B-Satz fields 18-22 — hot-water average temperature/consumption/flat-rate percentage/meter start-end) and *Versorgungszeitraum* (B-Satz fields 24-27 — up to two heating and two hot-water supply periods, start/end each) apply to both `bved_fuel_rule` values and are always shown (once `bved_fuel_data` is set) regardless of it.

‘meter’ (Zähler, B-Satz fields 28-34 — district heating or other supply metered directly, without a storable stock): `bved_meter_type` (BVED Tabelle ‘G’), `bved_meter_measurement_unit` (Many2One to `real_estate.bved.unit`, BVED Tabelle ‘E’ — the same model used for the M-Satz allocation keys above, reused here since it already covers the full Tabelle ‘E’ code list), `bved_meter_number`, `bved_meter_consumption`, `bved_meter_reading_start/bved_meter_reading_end`. `bved_primary_energy_factor` (field 34) is shown as part of this group, per the standard's field-position grouping, even though it is a property of the fuel type (mandatory for Fernwärme) rather than of the metering method itself. Hidden when `bved_fuel_rule = ‘stock’`.

All four groups are always packed into the B-Satz record regardless of `bved_fuel_rule` — the fields belonging to the currently hidden group are simply left empty by the user and packed blank/zero as usual.

Note

Every field label in this section (and the L-Satz Preview fields on `real_estate.bved.provider_assignment` below) is prefixed with its record field number, e.g. “10. Stock Start Date” — matching the convention already used by the M-Satz Preview line model. `bved_fuel_data` and `bved_fuel_rule` are left unnumbered since they are module-only controls, not fields of the B-Satz record itself.

Note

`bved_fuel_rule` did not exist before this field group split; a migration in `SettlementUnit.__register__` backfills it to 'stock' for every pre-existing row with `bved_fuel_data` set, so previously entered stock/hot-water data stays visible after the upgrade (it was, in effect, the only option before).

K-Satz cost records (`BvedExport._build_b_k_records()`) are built not only from invoice lines booked directly on the externally-billed settlement unit, but also from every settlement unit of the same billing unit that uses `allocation_via_cost_collector` with it as `reference_settlement_unit` (e.g. several fuel suppliers, each invoiced separately, feeding one heating settlement unit) — each such invoice line keeps its own settlement unit's type. `bved_cost_key` rather than the referenced unit's, since the underlying invoices may carry a different cost-type classification. B-Satz (fuel/stock data above) is unaffected — it stays tied to the externally-billed settlement unit itself.

Export (`real_estate.bved.export`, workflow Draft ☐ Generated ☐ Sent):

`provider` + `cutoff_date` (default: 31 December of the previous year — a still-running billing period shouldn't be exported yet since its cost data isn't final) default-select the eligible `billing_units` (assigned to this provider, `external_billing` = True, `end_date` <= `cutoff_date`); the list stays editable. `record_types` (multi-select A/L/M/B/K, default all five) controls which files get built.

generate

Validates every selected billing unit has a provider assignment matching this export's provider and a complete property address, auto-creates any missing object numbers, then builds one .DAT file per requested record-type group (A; L+M combined; B+K combined) and attaches them (ISO-8859-1 encoded, CRLF line endings) to the export record. Sets `export_date` and moves to generated.

mark_sent

Moves to sent — the actual transmission happens outside the app (manual file handoff, per the provider's `transport_type`).

All activity (files built, warnings such as a missing German IBAN for the A-/M-Satz bank fields) is appended to `activity_log`.

Import (`real_estate.bved.import`, workflow Draft ☐ Parsed ☐ Matched ☐ Processed):

Attach the provider's response file(s) as `ir.attachment` on the import record (filename prefix determines the record type: D/E835/ E898/P), then:

parse

Full fresh snapshot on every click: deletes every not-yet-applied line and rebuilds `real_estate.bved.import.line` rows from whatever is currently attached, so re-running after attaching a corrected/additional file never leaves stale duplicates. Already-applied lines are never touched.

match

Re-evaluates every line not yet applied (including previously error/skipped ones, so fixing a missing object number and clicking again re-checks them too). Resolves `internal_reference` + provider to a `bved.object_number` valid on the line's period end date, then the matching `settlement_result` for that object — cross-checked against the result's own billing unit's provider assignment, so an object with two concurrent externally-billed billing units (e.g. heating via one provider, water via another) cannot be matched to the wrong result. E835-/E898-/P-Satz lines missing "Letzter Tag Nutzungszeitraum" (a Mussfeld per spec for these three types) are flagged as an error rather than guessed. A line with no data and no match is skipped (not an error); one with data and no match is an error.

apply

Writes matched data onto the `settlement_result`:

- **D-Satz** — lines for the same result are grouped by `d_cost_key` (BVED Tabelle 'K' cost-type code): different keys are summed (e.g. Heizung + Warmwasser), two lines sharing the same key (including both empty) are treated as a duplicate delivery and only the last is kept, with a warning. Sets `actual_costs` (re-running `on_change_actual_costs()` so `refund_receivable` updates immediately), plus a plausibility check (total – advance $\approx$ balance per the D-Satz's own figures; reported advance vs. the internally computed `advanced_payment` — skipped when the reported sum is exactly 0, since BVED's fixed-width numeric fields can't distinguish "blank" from "genuinely zero"). Sets `bved_state` to `validated` or `validation_error` (with the concatenated messages in `bved_check_message`). Already-applied D-Satz lines whose result is still `validation_error` are re-evaluated on every subsequent Apply run too, so a plausibility-check fix (e.g. a corrected advance payment) takes effect without re-parsing.
- **E898-Satz** — copies the referenced PDF attachment (matched by filename) from the import onto the settlement result.
- **E835-/P-Satz** — informational only; no settlement-result field exists to write them back to, browsable via the result's own `bved_e835_lines/bved_p_lines` tabs.

Afterwards, logs which of a touched billing unit's covered objects still have no D-Satz result (`bved_state` not yet imported/validated/`validation_error`) as a completeness check.

line_summary

(Function.) A per-record-type/state line count plus amount summary (D-Satz total, E835/P-Satz user-share subtotal) shown on the import form.

real_estate.bved.import.line (bved.py)

One row per parsed fixed-width record, typed fields for all four import record types (D/E835/E898/P) plus `raw_line/raw_data` (full field dump as JSON) for audit. States: `parsed` ☐ `matched` ☐ `applied` (or `skipped/error`). Form notebook pages *D-Satz/E835-Satz/E898-Satz/P-Satz* show/hide automatically based on `record_type` (`view_attributes()`); the *Row* page (raw data) is always visible.

Fields added to `real_estate.settlement_result`:

bved_state

Selection (awaiting_import / imported / validated / validation_error) tracking the data-exchange progress independently of state (approved/billed); editable until billed, e.g. to manually clear a plausibility-check false positive.

bved_import_date / bved_check_message / bved_import_line

(readonly) — when the last D-Satz was applied, its check messages (if any), and the import line that last wrote actual_costs.

bved_matched_lines

(Function, One2Many.) Every import line matched to this result, regardless of type/state, shown as one unified, detailed table.

bved_e835_lines / bved_p_lines / bved_e835_labor_share_user_total / bved_p_user_amount_total

(Function.) Informational subsets of bved_matched_lines by record type, and their summed amounts.

Demo / test tooling. tests/test_bved_provider_response.py has no trytond/database dependency (it loads bved_records.py directly from disk by path) and simulates a provider's response from a set of exported A-/L-/M-/K-Satz files: pools K-Satz costs per property, distributes them across M-Satz units weighted by heating advance payment × occupancy-period length in days (with reproducible random jitter and a largest-remainder, cent-exact correction so the total still matches exactly), emits D-Satz plus, where applicable, E835-Satz (labor share) and a small synthetic P-Satz (Energiepreisbremse — format exercise only, the underlying support program has since ended), and one E898-Satz record per unit/period referencing a generated single-page PDF “Heizkostenabrechnung” (period, per-cost-type total vs. share, overall total/advance payment/balance) built with a dependency-free, hand-rolled minimal PDF writer:

```
python tests/test_bved_provider_response.py DTA310_....DAT DTM310_....DAT \
DTK310_....DAT [--out-dir DIR] [--seed N]
```

11.6 Option Rate (Input VAT Deduction)

Tracks, over time, what percentage of input VAT (Vorsteuer) on purchase invoices related to a real estate object is deductible (“optiert”) rather than treated as a final cost (added to the gross expense) — relevant wherever the property owner can opt for VAT liability on rent (§ 9 UStG). Applies to real_estate.base_object (property / building / land / rental object), real_estate.settlement_unit, and real_estate.billing_unit.

account.invoice.line.taxes_deductible_rate auto-fill (invoice.py)

For purchase invoice lines (invoice_type = 'in'), the core taxes_deductible_rate field (0–1 fraction of input VAT that is deductible; core already forces it to 0 when company.purchase_taxes_expense is set, via its own on_change_company) is auto-filled from the applicable option rate, divided by 100. Applies both interactively (on_change_base_object / on_change_settlement_unit / on_change_billing_unit) and to programmatic line creation (InvoiceLine.create() override) — the latter only fills the field when the caller did **not** pass taxes_deductible_rate explicitly, so an explicit value is never overridden.

The real-estate reference used is picked by InvoiceLine._option_rate_priority() in order of specificity: base_object □ settlement_unit □ billing_unit □ the derived property (as a base_object reference, last-resort fallback via on_change_with_property / term.property) — the first of these that is set on the line wins. OptionRate.get_current_rate_fraction(ref_field, record, date) then returns the record's latest option_rate at or before that date (taxes_date if set, else the invoice's invoice_date, else today), divided by 100; falling back to 1/0 for fix_100/fix_0 if no history record exists yet, or None (leaving the field untouched) for dynamic_measurement with no booked history.

The auto-fill is skipped entirely (field left at its core default of 1) when: the line is not a purchase line, or the company has `purchase_taxes_expense` set (core's own logic already governs that case). The line's contract field has no bearing on this logic and is ignored — a real-estate reference is used whenever present, even on a line that also carries a contract.

real_estate.option_rate (option_rate.py)

History-versioned option rate record, valid from a given date. Has three mutually exclusive Many2One reference fields — `base_object`, `settlement_unit`, `billing_unit` (all `ondelete='CASCADE'`) — exactly one of which must be set (enforced in `validate()`); a unique SQL constraint prevents two records for the same reference and `valid_from`. `option_rate` is a percentage (`digits=(5, 2)`, domain 0–100).

Fields added to `real_estate.base_object`, `real_estate.settlement_unit`, and `real_estate.billing_unit` (identical on all three):

option_rate_method

Required selection: `fix_0` (always 0 %), `fix_100` (always 100 %), or `dynamic_measurement` (calculated — see the wizard below). Defaults to `fix_0`. On a rental object (`base_object` with type = 'object'), changing `type_of_use` auto-sets it to `fix_100` for commercial and `fix_0` otherwise (`on_change_type_of_use`); `dynamic_measurement` is rejected by `validate_fields` for rental objects — only property/building/land/ settlement unit/billing unit may use it.

option_measurement_type

Many2One to `real_estate.measurement.type`, required only for `dynamic_measurement`. May reference a measurement **group**: all of its descendant leaf types are then summed per rental object (see the group hierarchy under `real_estate.measurement.type` above).

option_rates

One2Many (readonly) to the object's own `real_estate.option_rate` history.

purchase_taxes_expense

Function field mirroring the pre-existing `company.purchase_taxes_expense` field (added by `account_invoice`). When set on the object's company, all input VAT is always booked as an expense (fully gross) — equivalent to a permanent option rate of 0 % — and the *Option Rate* notebook page is hidden on the form (also always hidden for type = 'equipment').

Shown as a dedicated *Option Rate* page on the `base_object`, `settlement_unit`, and `billing_unit` forms (`page_option_rate`).

real_estate.option_rate_update.wizard (see Wizards below)

Batch recalculation entry point; see the *Wizards* section for the full selection/calculation logic.

Selection / filter screen and standalone list. Menu: *Real Estate* ▢ *Master Data* ▢ *Update Option Rates*, with a child menu item *Option Rates* opening a read-only list of all `real_estate.option_rate` records (both list and form views are fully non-editable — `creatable=""0"` plus `readonly=""1"` on every field; this applies only to the standalone list, not to the `option_rates` tab embedded in the `base_object/settlement_unit/billing_unit` forms, which remains editable there). A *Selektion* context form — modelled on the *Occupancy* context pattern (`real_estate.option_rate.context`, class `OptionRateContext`) — sits above the list with three optional filter fields, `base_object`, `billing_unit`, `settlement_unit`; whichever one is set narrows the list via `context_domain` (PYTHON, evaluated independently per field, so any combination — or none — can be applied).

11.7 Extensions to Core Modules

party.party (party.py)

Adds sequence (integer sort key) and salutation fields.

account.invoice (invoice.py)

Adds a contract Many2One field so invoices can be traced back to the originating contract. Shown on the invoice form's *Other Info* tab (view extension view/invoice_form.xml, inherits account_invoice.invoice_view_form).

_get_move_line(date, amount)

Core builds the invoice's receivable/payable move line directly from the invoice header via this method, bypassing InvoiceLine.get_move_lines() entirely (that one only copies contract etc. onto the regular revenue/expense lines, one per invoice line — see below). Without this override, the one line actually shown on the *Payable/Receivable Lines* list (account.type.receivable/payable) never got a contract, even though the invoice header itself has one. Overridden here to call super() then set line.contract = self.contract before returning — takes effect only for invoices posted from now on; already-posted invoices' receivable/payable lines cannot be corrected retroactively (core blocks writes to a posted move's business fields).

account.invoice.line (invoice.py)

Adds real-estate assignment fields, gated by assignment_control (Selection: *All*, contract, operating_costs, settlement_result_contract, settlement_result_vacant — controls which of the fields below are visible/required for a given line):

- contract / term — the originating real_estate.contract / ContractTerm
- base_object — the real-estate object the line relates to
- billing_unit / settlement_unit — for operating-cost billing lines
- property (Function) — derived from billing_unit/settlement_unit
- service_period_from / service_period_to — billed service period
- estg_35a (Selection) — German §35a EStG tax-deduction category (household services / craftsmen services)
- invoice_date, tax_amount, total_amount (Function fields)

Indexed on contract, term, settlement_unit, billing_unit.

account.move.line (invoice.py, class AccountMoveLine)

Mirrors the same real-estate fields as account.invoice.line above (assignment_control, contract, term, base_object, billing_unit, settlement_unit, property) directly on the journal entry line, so postings can be traced/filtered without going through the invoice line. Extended list view shows this context for journal entries.

effective_contract (Function field)

Getter on_change_with_effective_contract, searcher search_effective_contract. This line's own contract if set, else the contract of whichever *other* line in the same reconciliation group has one. Needed because a payment line never gets its own contract at booking time (Invoice.get_move_line() below only sets it on the invoice's own receivable/payable line) — once that line is reconciled against the payment, effective_contract lets the payment line be found too when filtering/searching by contract, without writing anything onto the (typically already posted) payment line itself. The stored contract field is left untouched either way. search_effective_contract expands to ['OR', ('contract', ...), ('reconciliation.lines.contract', ...)] — a plain dotted domain through the O2M reconciliation.lines, no custom SQL needed.

Payable/Receivable Lines (act_contract_move_line_payable_receivable in contract.xml)

Menu *Contracts*, between *Contract* and *Create Contract Moves*. A standalone equivalent of the core party.party form's own *Payable/Receivable Lines* relate action (act_move_line_payable_receivable in the account module, account.move.line.receivable_payable.context) — same underlying list (open, i.e. unreconciled, lines on a receivable- or payable-type account, account.move.line) but reachable directly from a menu instead of requiring an already-open party record — the core action's own domain is fixed to ('party', 'in', Eval('active_ids')), which only resolves once a specific party is already open.

order: [('payable_receivable_date', 'ASC NULLS FIRST'), ('move', 'ASC')] — sorted by the “Datum” column first, then by “Buchungssatz” (move) as the tiebreaker within the same date (core's own action instead falls back to ('id', 'DESC') for that second key).

Tree view: contract_move_line_view_list_payable_receivable (view/contract_move_line_payable_receivable_list.xml) inherits the core account.move_line_view_list_payable_receivable — none of the changes below touch the core view itself, only this module's own copy:

- party becomes optional="1" (core's own copy shows it unconditionally) — needed here since, unlike the party-form relate action, this list isn't already scoped to one party, so hiding the column is a real, useful option once a specific party/ contract filter is applied.
- payable_receivable_date (maturity_date if set, else date — core's own “Date”/“Datum” column) is relabelled “Fälligkeitsdatum” and made optional="1"; a second column for the plain date field (“Buchungsdatum”) is added right after it, also optional="1" — so both the due date and the booking date can be shown side by side, or either hidden, instead of only the due-date-if-set column core shows.
- amount becomes optional="1" (core always shows it). Right after it, debit/credit (German “Soll”/“Haben”, already existing account.move.line fields — a line's debit and credit are always mutually exclusive, so no derivation from the signed amount column is needed) are added, both optional="1".
- payable_receivable_balance is relabelled “Saldo” (matching core's own override, kept here since the field node was replaced for the optional="1" addition) and made optional="1" (core always shows it).

Filter panel (real_estate.contract.move_line_payable_receivable.context, own model, not an extension of the core context — the core one has no contract/account/date-range fields to add to without also changing the party form's own relate dialog):

- company (required)
- party, contract — filters via effective_contract (see account.move.line above), not the plain stored contract field, so a reconciled payment line is found too, not just the invoice's own receivable/payable line — account — all optional, exact match
- date_from/date_to — restrict account.move.line.date (the line's own Effective Date)
- receivable/payable/reconciled — same three checkboxes, same defaults (True/True/False, also readable from the action context, e.g. when reached from elsewhere with receivable/payable/reconciled pre-set) and the same context_domain shape as the core context's own account.move.line.receivable_payable.context (each unchecked toggle exclusion falls back to ('id', '<', 0) — an always-false clause — inside its OR branch, and reconciled unchecked adds ('reconciliation', '=', None)).

Print (real_estate.contract.move_line_payable_receivable.report)

Class ContractMoveLinePayableReceivableReport in invoice.py, template report/

`contract_move_line_payable_receivable_de.odt`. Registered like any other report in this module (`ir.action.report + an ir.action.keyword` with `keyword='form_print'` on `account.move.line,-1`) — select the desired rows in the list (e.g. “Select All” after applying the filter panel above) and use the client’s *Print* action. Not tied to the list’s own live column/sort choices, since those are client-side UI state the report engine cannot see: the printed table always shows a fixed column set — Datum (`payable_receivable_date` — `maturity_date` if set, else `date`, matching the list’s own “Datum” column exactly; using the plain `date` field here would print the booking date instead of the due date shown in the list whenever a line has its own `maturity_date`), Partei, Soll, Haben, Buchungssatz (move), Beschreibung (`description_used` — the line’s own description, falling back to its origin’s, via the `DescriptionOriginMixin` already used by core) — plus a totals row (Soll/Haben summed).

Note

The filter panel’s own `date_from/date_to` still restrict the plain `date` field (see above), not `payable_receivable_date` — a related, but distinct, inconsistency between the list/report’s *display* field and the *filter*’s field, left as-is since it wasn’t reported as a problem; flag if it should also switch to `payable_receivable_date` for full consistency.

`get_context()` adds lines (the selected records), `total_debit/total_credit` (summed), and the same `format_value` helper (date/number formatting) used by `ContractReport/ContractAnnex4Report`, extended here with an explicit `Decimal` branch (the existing helper only handled `float`, needed for debit/credit) and now passes `digits=2` explicitly, so amounts always show exactly two decimal places regardless of the underlying `Decimal`’s own precision.

Note

Naming fix: this helper and its template placeholder were originally named `_format` (leading underscore). Trytond’s report rendering runs Genshi templates through a `SafeASTTransformer` (`trytond/_safe_genshi.py`) that rejects any bare name starting with `_` (except a small allow-list, e.g. `__class__`) to keep templates from reaching private/dunder attributes — so `py:content="_format(...)"` raised `ValueError: invalid name '_format'` the moment the report was actually printed (`Report.execute()`), even though the module installed and all module-consistency tests passed (they never render a template). Renamed to `format_value` throughout (Python class and `content.xml`) to fix this for both this report and the *Kontenblatt* one below, which reuses the same pattern. Lesson: any context variable/helper exposed to a Genshi report template must not start with an underscore.

`_selection_summary()` builds a one-line German summary of the filter panel’s own values (“Partei: ...; Vertrag: ...; Konto: ...; Zeitraum: ... - ...; Forderungen, Verbindlichkeiten; inkl. ausgeglichene Posten”) from `Transaction().context` — printed near the report header (`selection_summary` in the template) so the applied filter is visible on the printout itself, not just on screen. This only works because `act_contract_move_line_payable_receivable` (unlike the plain party-form relate action, and unlike most other `context_model`-based actions in this codebase) also declares its own context field forwarding every `real_estate.contract.move_line_payable_receivable.context` field into the transaction context — `context_domain` alone only uses those values to build the search domain client-side, it does **not** by itself make them available server-side for anything else (like this report). If a value was never set in the panel, its section is simply omitted; with nothing set at all the summary

reads “keine Einschränkung”.

Note

The template’s own formatting (10pt table font, right-aligned Soll/Haben, per-column widths — narrow throughout except a wide Beschreibung column) is defined via `<office:automatic-styles>` directly inside `content.xml` (PHead10/PHead10Right/PContent10/ PContent10Right paragraph styles, Col* table-column styles) rather than by editing the `.odt` in LibreOffice: LibreOffice does not recognize the `py:` (Genshi) namespace used for the template’s placeholders and drops those attributes entirely on save, silently breaking the report. Any further layout change to this report should go through `content.xml/styles.xml` as text, never through opening and re-saving the `.odt` itself in an office suite.

Kontenblatt (act_contract_general_ledger_account_contract in contract.xml)

Menu *Contracts*, between *Payable/Receivable Lines* and *Create Contract Moves*. A standalone equivalent of the `real_estate.contract` form’s own *Party Ledger* (“Kontenblatt”) button (`Contract.open_party_ledger()` in `contract_core.py`, opening `act_general_ledger_account_contract_form_contract` — a `form_relate` action whose domain is fixed to `('contract', 'in', Eval('active_ids'))`, so it only ever shows the already-open contract’s own rows). Same underlying model (`real_estate.account_contract` — per-(account, party) aggregated balance/debit/credit for the contract’s own party, joined to `real_estate.contract` via `contractual_partner` in `GeneralLedgerAccountContract.table_query()`), reachable directly from a menu, with no `active_ids`-dependent domain and no fixed contract restriction — matching how core’s own analogous standalone *General Ledger - Accounts* menu entry (`act_general_ledger_account_form`) works. Carries over the same `search_value` default as that core action (`[('line_count', '!=', 0)]` — hides accounts with no activity).

context_model: `real_estate.contract.general_ledger_account_contract.context` (`ContractGeneralLedgerAccountContractContext` in `contract_core.py`) — a Python subclass of core’s own `GeneralLedgerAccountContext` (`account.general_ledger.account.context`), inheriting its `fiscalyear/period/date-range/company/posted/journal` fields and defaults unchanged, and adding two extra filter fields on top: `party` and `contract`. Registered under its own model name (not by extending core’s context model directly) so that other, unrelated General Ledger views keep using core’s context model as-is. The `act_window`’s `context_domain` translates these two extra fields into a domain on `real_estate.account_contract` (which already exposes `party/contract` as plain columns): `If(Eval('party', None), [('party', '=', Eval('party', None))], [])` and the analogous clause for `contract` — an empty clause `[]` when the field is left blank, so leaving both empty behaves exactly as before (no restriction beyond `search_value`). **View:** `contract_general_ledger_account_contract_context_form.xml` (a full, standalone form — not an `inherit` of core’s own context form, since inheriting requires the same model — with `party` and `contract` added above the unchanged core fields).

Tree view: `contract_general_ledger_account_contract_view_list` (`view/contract_general_ledger_account_contract_list.xml`) inherits `general_ledger_account_contract_list` and adds the `contract` column (`optional="1"`) right after `party` — present but commented out in the base view (redundant there, since every row is always the one already-open contract; useful here since rows for every contract are mixed together in one list).

Note

Company scoping fix in `AccountContract.table_query()` (`real_estate.contract.account_contract`, `contract_core.py`) — a pre-existing gap only exposed once this standalone menu removed the implicit company-scoping that the `form_relate` button always had via its `active_ids`-restricted domain. The (account, party) grouping's join to `real_estate.contract` (to populate the contract column) previously matched on `contractual_partner` alone, with no company filter anywhere in the query. Once reachable without an `active_ids` restriction, this could return rows from other companies, and — worse — yield *more than one* joined contract row for the same (account, party) pair whenever that party has more than one contract (a follow-up contract after termination, or contracts in different companies), breaking the SQL view's id uniqueness and raising `RuntimeError: Undetected access error on read`. Fixed by (1) filtering the query by `account.company == context.get('company')`, matching core's own `_GeneralLedgerAccount.table_query()` pattern, and (2) joining against a grouped subquery of `real_estate.contract` (one row per `contractual_partner`, restricted to the current company, `Max(id)` picking the most recent contract when several match) instead of the raw table directly, guaranteeing at most one match per party. Both the pre-existing `form_relate` button and this new standalone menu use the same underlying model and benefit from the fix.

Print (`real_estate.contract.general_ledger_account_contract.report`)

Class `ContractGeneralLedgerAccountContractReport` in `contract_core.py`, template `report/contract_general_ledger_account_contract.de.odt`. Registered the same way as the Payable/Receivable report above (`ir.action.report + an ir.action.keyword` with `keyword='form_print'` on `real_estate.account_contract,-1`) — select the desired rows in the *Kontenblatt* list and use the client's *Print* action. Fixed column set (independent of the list's own live column/sort choices): Konto, Partei, Vertrag, Anfangssaldo, Soll, Haben, Endsaldo — plus a totals row (all four numeric columns summed, matching the list's own `sum="1"` columns).

Same `format_value` helper and `get_context()/_selection_summary()` pattern as the Payable/Receivable report (see above, including the leading-underscore naming pitfall this new report deliberately avoided from the start). `_selection_summary()` here reflects the *Kontenblatt* context model's own fields instead (Partei, Vertrag, Geschäftsjahr, Periode, Zeitraum, Journal, “nur gebuchte Bewegungen”) — available server-side only because `act_contract_general_ledger_account_contract` declares an explicit context field forwarding all ten `ContractGeneralLedgerAccountContractContext` fields into the transaction context, on top of them already feeding `context_domain` (for party/contract) resp. being read directly via `Transaction().context.get(...)` inside `table_query()` (for the core `fiscalyear/period/date/company/ posted/journal` fields, unaffected by `context_domain`).

Note

Without that explicit context field, the generic `test_ir_action_window` module-consistency test caught an unrelated but real bug during development: `pyson.Eval(name)` without an explicit default decodes to an empty string (`''`), not `None`, whenever the variable is absent from the evaluation context. `account.general_ledger.account.context's default_fiscalyear()` then does `fiscalyear_id >= 0` on that empty string and raises `TypeError`. Fixed by giving every `Eval(...)` in this action's context field an explicit `None` (`False` for posted) default. The pre-existing Payable/Receivable action's own context field has the same latent gap, left unchanged here since it happens to be harmless there (`account.move.line` never reads those keys inside a `table_query()`, unlike this

General-Ledger-based model).

account.general_ledger.line (invoice.py, class GeneralLedgerLine)

Adds contract, term, base_object, billing_unit, and settlement_unit (all readonly Many2One) to Tryton's standard General Ledger — Lines report (German: *Kontenblätter – Positionen*), so the individual postings for an account can be filtered/traced back to the originating real-estate contract, term, object, billing unit, or settlement unit. No override of table_query is needed: it pulls any non-Function field straight from the account.move.line table by matching field name, and account.move.line already carries these same fields (see above). Shown as optional columns in the tree view (view/general_ledger_line_list.xml, inherits account.general_ledger_line_view_list).

account.configuration (account_configuration.py)

Extends the standard account configuration with real-estate-specific defaults used during operating cost billing:

re_account_allocation_by_owner

Vacancy cost account (debit and credit side of vacancy postings).

re_journal_billing

Journal used for direct GL postings in vacancy settlements.

re_payment_term_billing

Default payment term for operating cost settlement invoices. Pre-fills the payment_term field of the real_estate.billing_unit.wizard; also used directly by BillingUnit.billing() as a final fallback when neither the wizard's payment_term nor the contract's own payment term is set.

All three are per-company MultiValue fields backed by account.configuration.real_estate.

res.user (res.py)

Adds phone and mobile fields.

WIZARDS

`real_estate.contract.create_moves.wizard` (`contract_wizard.py`, class `CreateContractMovesWizard`)

Batch invoice generation wizard. Supports three actions:

`create`

Generate invoices up to a given date. All `ContractTermCashFlow` entries created in this run share the same `create_moves_run_id` (YYYYMMDD-HHMMSS-U<userid>).

`re_calc`

Rebuild cash flow projections without creating invoices.

`re_calc_and_create`

Combine both steps.

Can be filtered by property and/or contract list. Optional queue execution via `execute_in_queue`.

`real_estate.billing_unit.wizard` (`billing_unit_wizard.py`, class `BillingUnitWizard`)

Batch billing wizard, opened via the `billing_wizard` button on the billing unit form or the `billing_property` button on the property form.

Start — `date` (default: last day of the current month), `company`, `invoice_date` (default: today; auto-set to the first of date's month when date lies in the past), `invoice_state` (draft / posted — invoices are posted immediately when posted), `payment_term` (default: account configuration's `re_payment_term_billing`; written to every invoice created by the run and therefore drives its due date — takes priority over the contract's own payment term), `execute_in_queue` (default `True`), optional `property`s / `billing_units` filters (pre-filled from the active record when launched from a property or billing unit form/list).

Confirm — read-only summary of the matching billing unit count and the selected filters before processing.

Process (`transition_do_billing`) — calls `BaseObject.call_billing` for the resolved properties, which invokes `BaseObject.do_billing` either directly or via the background queue. Only billing units already in state `ready_for_billing` are billed; with `collective_billing` all billing units of a property sharing the same start date must be included, otherwise a `ValidationError` is raised.

Result — reports how many billing units were queued or processed.

`real_estate.terminate_contract.wizard` (`contract_wizard.py`)

Sets a contract to *Terminated*, records `terminated_by`, `receipt_of_termination_notice`, `termination_reason`, and calculates `termination_date` from the notice period (3 / 6 / 9 / 12 months to end of month); also writes the resolved `termination_date` into the contract's `end_date` (see *Fixed term / termination* under `real_estate.contract` above). `terminated_by_type` is `tenant` or `landlord` for a manual termination via this wizard; see the `update_contract_status`

cron task above for the automatic expired case (fixed-term contracts with no active termination), and the *Revert Termination* button to undo a termination.

real_estate.estimate_consumption.wizard (base_object.py, class EstimateConsumptionWizard)

Two-step wizard launched from the *Meters* tab of any approved meter object. Estimates the meter reading for a chosen cut-off date and books it as an estimate reading.

Step 1 – Start: displays the meter description, unit and factor, pre-fills meter_id from the last reading, and prompts for per_date (default: today) and a free-text reason.

Step 2 – Result: calls simulate_estimate and shows the two basis readings (reading1 / reading2), the derived consumption, and the editable estimated_value. The user can adjust the value before booking.

Book: saves a new MeterReading with m_type = 'estimate'. The value is rounded to the meter's UOM digit precision (integer if meter_no_decimals is set) before saving.

real_estate.option_rate_update.wizard (option_rate_wizard.py, class OptionRateUpdateWizard)

Recomputes and books option rates (see *Option Rate* above) for a selection of base objects, billing units, and/or settlement units, as of a cut-off date.

Start — base_objects (Many2Many, restricted to property/building/land/rental object; an approved property/building/land is expanded to its building/land/rental-object descendants, and an approved property additionally cascades to its billing units and their settlement units), billing_units (expanded to their settlement units), settlement_units, cutoff_date (default: today — booked option rates are always dated on the first of this month).

Confirm — read-only counts of the directly selected base objects / billing units / settlement units, and the total number of objects that will actually be processed after expansion (n_expanded).

Process (OptionRate.process_update) — for every expanded object:

- Objects with option_rate_method fix_0 / fix_100 are booked at 0 % / 100 % directly, without looking at any sub-objects.
- dynamic_measurement objects are calculated from their approved rental objects: for a rental object, itself; for a property/building/ land, its approved rental-object descendants regardless of how many buildings/land lie in between (a non-approved intermediate object excludes its whole subtree — both from booking and as a calculation basis); for a settlement unit, its existing objects function field (i.e. whatever it already resolves to via its own allocation rule/regex — read-only here, never modified by this wizard); for a billing unit, the union of objects across all its settlement units that are not draft/billed, deduplicated. Each contributing rental object is weighted by its measurement value (leaf types under option_measurement_type summed if it is a group) at the cut-off date, and contributes its own current option rate (its latest option_rate record at or before the cut-off date, else its own fix_0/fix_100 default). The new rate is the resulting weighted average, rounded to 2 decimals.
- Base objects not in state approved, and billing/settlement units in state draft or billed, are excluded entirely.
- A new dated option_rate record is only written when the computed rate differs from the current one; a record already dated on the cut-off month's first day is updated in place rather than duplicated.

Result — counts of created / updated / unchanged / skipped records, plus a per-record detail log.

Menu: *Real Estate* ▢ *Master Data* ▢ *Update Option Rates*.

real_estate.contract_term_adjustment.wizard (contract_wizard.py, class ContractTermAdjustmentWizard)

Note

Skeleton only. The *Start* → *Confirm* → *Process* → *Result* flow and every field below already work end-to-end, but the actual selection of contracts/terms and the write-back to `ContractTerm / real_estate.contract.term.adjustment` are not implemented yet. `transition_do_adjustment` dispatches by procedure to one of three placeholder methods (`_adjustment_operation_costs_billing`, `_adjustment_operation_costs_plan`, `_adjustment_free_adjustment`), each of which currently just returns `processed = 0` and a “not yet implemented” message without changing any data.

Start — procedure (`operation_costs_billing / operation_costs_plan / free_adjustment, required`), `company`, `property` (defaulted from the active property record when launched from a property form/list), `valid_from_new` (effective date of the replacement term — the old term is intended to close the day before). Only for `free_adjustment`: `adjustment_mode` (`percentage / absolute`). Only for `operation_costs_billing`: `billing_run_id` (Selection, populated from billed billing units matching company/property), `guard flags no_terminated_contracts, no_future_terms, no_booked_terms, only_rhythm_monthly_1` (all default `True`), and caps `max_adjustment_percent` (default 10 %) / `max_adjustment_absolute` (default 50, in the company currency).

Confirm — read-only echo of every *Start* value (no match count, since selection logic does not exist yet).

Result — processed count and a details message.

Menu: *Real Estate* → *Contracts* → *Adjustment* (submenu with the wizard itself and a read-only list of `real_estate.contract.term.adjustment` records).

REPORTS

ODT templates (OpenDocument Text) are located in `report/` and rendered via Genshi/relatorio; `template_extension` on the `ir.action.report` record is `odt`. Values are inserted with `text:span py:content="..."` rather than literal `${...}` interpolation (which relatorio escapes in ODT text), and control flow (`py:if/py:for/py:choose`) is written as attributes on the surrounding ODF elements. Superseded `.html` versions of these templates remain in `report/` for reference but are no longer registered.

`real_estate.contract.report` (`contract_report.py`)

General contract report. Templates: `contract_en.odt`, `contract_letter_de.odt`.

Note

Two genuine, pre-existing bugs found and fixed while implementing the termination confirmation report below (both had always been broken — module-consistency tests never render a template, so neither was ever caught):

1. The context helper was named `_format` (leading underscore). Trytond's report rendering runs Genshi templates through a `SafeASTTransformer` (`trytond/_safe_genshi.py`) that rejects any bare name starting with `_` — so `py:content="_format(...)"` raised `ValueError: invalid name '_format'` the moment `contract_letter_de.odt` was actually printed. Renamed to `format_value` throughout (also in `ContractAnnex4Report/anlage4_contract_de.odt`, which had the same issue).
2. `contract_letter_de.odt` additionally had a literal `from decimal import Decimal` inside its `<?python?>` block (needed for `sum(..., Decimal(0))`). The same `SafeASTTransformer` unconditionally forbids *any* import statement, raising `ValueError: invalid import from` — independently of the `_format` issue, and would have kept the report broken even after fixing that. Fixed by injecting `Decimal` via `get_context()` instead (the same pattern core itself uses for `datetime`), removing the import from the template entirely. Also added a `Decimal` branch to `format_value()` (previously missing, so `total_amount` would have rendered as Python's own `str(Decimal(...))` instead of a properly formatted German amount once the import itself was fixed).

`real_estate.contract annex4.report` (`contract_report.py`)

Annex 4 – Betriebskostenaufstellung. Template: `anlage4_contract_de.odt`. Renders grouped operating cost positions with BetrKV paragraph references and allocation method labels, sourced from the contract's own `settlement_units` field (see `real_estate.contract` below) rather than re-deriving the billing unit independently. Allocation labels are translatable messages (`real_estate.msg_allocation_*` in `message.xml`).

`real_estate.contract.termination.confirmation.report` (`contract_report.py`)

Termination confirmation letter ("Kündigungsbestätigung"). Template:

contract_termination_confirmation_de.odt. Registered exactly like the Contract Letter/Annex 4 reports above — an `ir.action.report` plus an `ir.action.keyword` with `keyword='form_print'` on `real_estate.contract,-1` — so it appears in the standard Print toolbar for every contract, not behind a dedicated button. “Only available once a termination is recorded” is therefore enforced entirely inside `get_context()`: it checks `record.state == 'terminated'` for every selected record and raises `msg_contract_termination_confirmation_not_terminated` (a clear, translated error) otherwise, rather than by hiding the menu entry itself. Resolves the translated labels for `terminated_by_type/termination_notice` (both Selection fields) via `Contract.fields_get()` rather than exposing the raw storage keys to the template.

`real_estate.base_object.report (base_object.py)`

Fact sheet for a property or object. Template: `fact_sheet.odt`.

ACCOUNTING / WOWI

The `wowi/` directory contains a German “Kontenrahmen der Wohnungswirtschaft” (WoWi, housing-industry chart of accounts) as XML templates loaded at module installation:

- Account types and accounts
- Tax groups, tax templates, tax code templates, tax code line templates
- Tax rule templates

https://www.intex-publishing.de/pdf/Kontenrahmen_Wohnungswirtschaft.pdf
https://www.intex-publishing.de/pdf/Kontenrahmen_Immobilienwirtschaft.pdf

https://www.intex-publishing.de/pdf/Kontenrahmen_Immobilienwirtschaft.pdf

SOURCE LAYOUT

```
real_estate/
├── address.py                # real_estate.address
├── base_object.py           # real_estate.base_object, occupancy, meter_
├── readings
├── billing_unit.py          # real_estate.billing_unit, billing_unit.moves,
                             #   billing_unit.log, cost_type,
├── cost_category_group
├── billing_unit_wizard.py    # real_estate.billing_unit.wizard (batch billing)
├── bved.py                  # BVED Tryton models: service_provider,
├── provider_assignment,      #   object_number, export, import(.line)
                             # BVED fixed-width record (de)serialization, no_
├── bved_records.py          # real_estate.co2_kostaufg(.consumption),
├── Tryton dependency
├── co2_kostaufg.py           # extension to company.company (re_accounting_
├── co2_emission_share       # link)
├── company.py               # real_estate.contract, contract.log, account_
├── link
├── contract_core.py         #   cron_daily dispatcher + cron task handlers
                             # real_estate.contract.item, contract.item.object
├── contract_item.py         # real_estate.contract.term, cash_flow,
├── contract_term.py         # Quantitative
├── contract_type.py         # real_estate.contract.type, term.type
├── contract_report.py       # ContractReport, ContractAnnex4Report
├── contract_wizard.py       # wizards (CreateContractMovesWizard,
├── TerminateContractWizard, ...)
├── cron_task.py             # real_estate.cron_task (per-company scheduled_
├── task config)
├── invoice.py               # extensions to account.invoice / invoice.line
├── ir.py                    # extension to ir.cron (registers the cron_daily_
├── method)
├── measurement.py          # real_estate.measurement.type, measurement
├── object_party.py          # real_estate.object_party, object_party.role
├── option_rate.py           # real_estate.option_rate, option_rate.context
├── option_rate_wizard.py    # real_estate.option_rate_update.wizard
├── party.py                 # extension to party.party
```

(continues on next page)

(continued from previous page)

```
|— re_accounting.py      # real_estate.re_accounting (company-scoped RE_
↳config)
|— res.py                # extension to res.user
|— settlement_result.py  # real_estate.settlement_result, cost_share
|— settlement_unit.py    # real_estate.settlement_unit
|— report/              # ODT report templates (rendered via_
↳Genshi/relatorio)
|— view/                # XML form and tree view definitions
|— wowi/                # German WoWi accounting templates
|— locale/              # Translations (de.po)
```

RUNNING TESTS

Warning

tests/ currently has no unittest.TestCase-based tests — test_module.py (the standard Tryton ModuleTestCase boilerplate) has been removed and not replaced. tox.ini still defines a real test matrix (envlist = {py39,py310,py311,py312,py313}-{sqlite,postgresql}) and the commands below run without error, but unittest/xmlrunner discovery currently finds **zero** test cases in every environment and exits **0 (success)** regardless — i.e. tox reports a false-positive pass, not “no tests configured”. There is no CI pipeline in this repo currently invoking it automatically, but running it manually and reading “OK” is misleading until a real test case is added back.

```
# Full test matrix (sqlite + postgresql, py39–py313) – currently a
# false-positive pass, see warning above
tox

# Single environment
tox -e py311-sqlite

# Direct run
export TRYTOND_DATABASE_URI=sqlite://
export DB_NAME=:memory:
coverage run --omit=*/tests/* -m xmlrunner discover -s tests
coverage report
```

16.1 Demo data scripts

Everything in tests/ is instead a one-shot **proteus import script** that generates demo data against a running trytond server. These are run manually — python tests/<script>.py --database <db> [--config trytond.conf] — not via tox/xmlrunner, in the following order (each step builds on the previous one’s data):

tests/test_immo.py

No prerequisites. Creates two properties (*Musterstraße 1-4* and *Musterstraße 5-8*), each with four buildings. Every building has one ground-floor retail unit (type_of_use='commercial', 140 m²) and four apartments (type_of_use='residential', 57/83 m² alternating) — 16 apartments, 4 retail units, and 20 water meters per property in total — plus one land parcel with four parking spaces. Meter names follow the pattern Wasser Zähler NN / Wasser Zähler EHNN, each with an initial and a consumption reading. Looks up real_estate.use_class by sequence (language-independent) and measurement types by German name — the admin user must be set to German:

```
python tests/test_immo.py --database <db> [--config trytond.conf]
```

tests/test_contracts.py

Requires `test_immo.py`. Creates residential lease contracts for 15 of 16 apartments per property (one left vacant), each with rent, operating cost, and heating terms; three contracts per property are terminated, two of them with a follow-up contract. The four parking spaces per property are split between existing apartment contracts (as an extra contract item) and new standalone parking contracts. Also creates commercial lease contracts for the retail units — one combined contract for all four retail units on *Musterstraße 1-4*, one contract per unit on *Musterstraße 5-8*:

```
python tests/test_contracts.py --database <db> [--config trytond.conf]
```

tests/test_billing_unit.py

Requires `test_immo.py`. Creates two billing units per property: *Kalte Betriebskosten* (measurement- and consumption-based settlement units, vacancy allocated to the owner) and *Heizkosten* (external_billing=True, settlement units use allocation_from_external_billing). Billing units are left in state draft — anything that requires a later state (e.g. `test_invoices.py`'s settlement-unit lookup) needs the workflow advanced manually first:

```
python tests/test_billing_unit.py --database <db> [--config trytond.conf]
```

tests/test_kreditor.py

No prerequisites. Creates 11 supplier/creditor parties (property tax office, building insurer, utilities, cleaning, caretaker, gardening, chimney sweep, heating maintenance), each with an invoice/delivery address:

```
python tests/test_kreditor.py --database <db> [--config trytond.conf]
```

tests/test_invoices.py

Requires `test_kreditor.py`, `test_immo.py`, and `test_billing_unit.py` (with billing units advanced past draft so settlement units can be looked up). Creates the corresponding purchase invoices — one-off and recurring — for each property, linking each line to its settlement_unit where a match is found. Invoices are only saved, not posted (remain in state draft):

```
python tests/test_invoices.py --database <db> [--config trytond.conf]
```

tests/test_payment.py

Requires `test_contracts.py` and at least one run of the CreateContractMoves wizard so posted tenant invoices exist. Books one payment receipt per tenant/commercial-tenant party (debit account 1800 Bank / credit the receivable account taken from that party's open lines) and reconciles the open items:

```
python tests/test_payment.py --database <db> [--config trytond.conf]
```